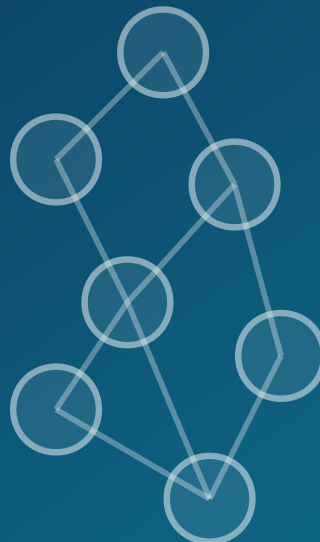


个人事务运行系统

语义、工作流、Agent 与 Logseq 插件一体化设计报告

从自由记录出发，通过可审查的语义加工，形成可执行、可追溯、可重入的个人事务系统。



执行摘要：这不是待办清单，而是一套个人事务运行系统

当工作、学习和 Vibe Coding 项目不断增加时，真正困难的并不是“有没有记录”，而是信息以怎样的成本进入系统、如何获得正确语义，以及中断后能否迅速恢复行动能力。现有 Logseq 工作区之所以越来越复杂，是因为原始记录、项目说明、任务数据库、工作流、历史档案和当前工作台被迫共享同一批块与字段。

核心命题

记录时尽量自由，审查时拥有控制，工作时只看必要信息，回顾时能够看见完整证据。

25

主体规范文档

145

稳定规则

20

端到端场景

12

黄金样例

10

MVP 验收门槛

从模糊输入到可执行事务：把整理成本放在可审查的中间环节



记录时自由 · 审查时透明 · 工作时克制 · 历史可追溯

图 1 系统将结构化成本放在“Agent 建议 - 用户审查 - 原子提交”的中间环节。

十项关键设计决定

这些决定共同解决自由记录与正式 workflow、完整信息与克制阅读、Agent 自动化与用户控制之间的矛盾。

1 自然语言优先

人通过叙事理解事务；结构化字段负责计算，但不得取代必要的背景与因果。

2 渐进式正式化

输入可以先模糊存在，重要内容再经过分诊、审查和正式化获得结构。

3 正文唯一、历史可追溯

批准后的正式内容替换原文，避免双版本污染；原始版本进入审计历史。

4 建议可拆分

文字、类型、归属、位置、状态和对象创建是不同决策，必须能够分别接受。

5 位置不等于语义

Journal 中的正文可以属于某个 Project；物理移动不能被当作改变归属的唯一方式。

6 三轴状态模型

生命周期、执行条件与注意力信号分离，准确表达 ACTIVE + WAITING + REVIEW_DUE。

7 单类信息单一权威

Logseq 管正文，领域模型管状态与关系，审计层管历史，视图可以重新生成。

8 投影视图而非巨型页面

执行、重入、等待和周审查读取同一数据，但默认显示完全不同的信息。

9 Agent 处理不确定语义

LLM 负责理解、归属候选和摘要；唯一性、流转、提交与回滚由代码保证。

10 领域模型独立于 Logseq

Logseq 是首要入口，但未来 CLI、Web 或其他笔记前端应共享同一语义骨架。

规范不是并列文档，而是一条向下传递、向上验证的约束链



图 2 规范体系从目标与语义向实现传递约束，再由场景与测试反向验证。

规范覆盖概览

145 条规则并非平均分布。关系、原则、对象语义与生命周期构成领域骨架；审查、文字、视图和测试负责把骨架变成可使用、可验证的系统。

规则分布：最密集的十二个领域

关系与语义规则最多，说明项目优先解决“系统中的世界是什么”而不是先堆界面功能。

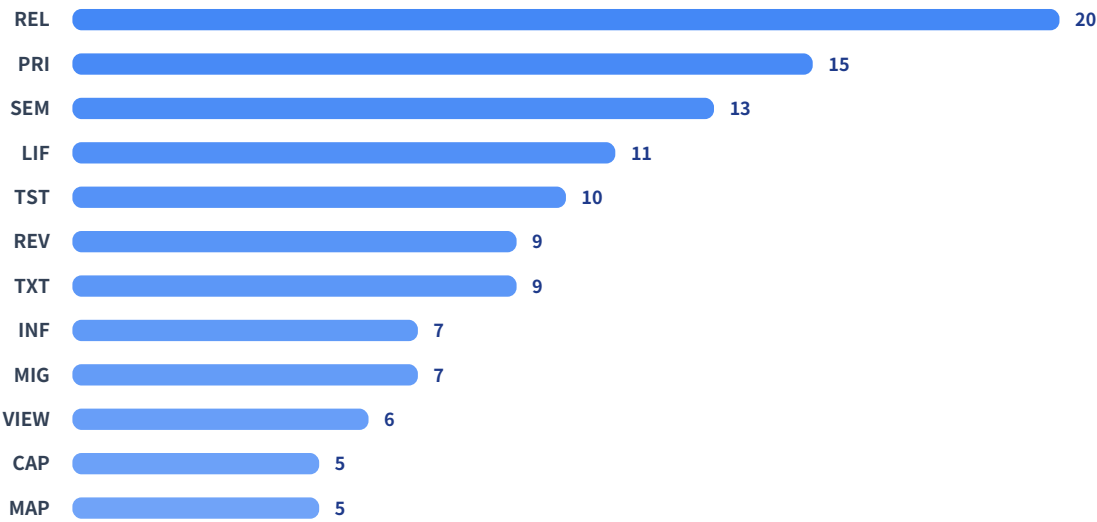


图 3 规则数量反映规范关注度，不代表实现工作量。

阅读提示

正文中带有彩色规则编号的卡片属于规范性约束；普通段落负责解释设计理由与使用方式。**MUST** 表示强制，**SHOULD** 表示默认遵守，**MAY** 表示允许但非必需。

目录

本报告先建立整体心智模型，再进入五个相互咬合的规范部分。目录页码由排版引擎自动生成。

PART I · 愿景与领域语义

01	项目愿景、边界与不可变原则	9
02	核心概念与术语表	14
03	事务对象语义与契约规范	20
04	关系、归属与层级规范	29
05	生命周期与状态机规范	35
06	对象迁移、拆分与合并规范	41

PART II · 信息、提交与审查

07	信息分层与权威来源规范	46
08	Logseq 映射与对象锚点规范	50
09	版本、审计与回滚规范	54
10	捕获、分诊与正式化流程规范	57
11	Agent 语义建议与提交协议	61
12	审查与部分接受规则	67

PART III · 注意力、呈现与 Agent

13	注意力场景与视图投影规范	72
14	自然语言呈现与信息密度规范	77
15	结构显隐与渐进披露规范	81
16	Agent 角色、权限与自动化边界	85
17	Skill 架构与编写规范	89

PART IV · 架构、插件与外部资源

18	总体系统架构与领域数据模型	94
19	Logseq 插件与适配层设计	101
20	文件工作区与外部资源集成规范	106
21	同步、冲突、备份与恢复规范	109

PART V · 场景、验证与治理

22	核心使用场景目录	114
23	黄金样例与反例库	120
24	验收标准、测试策略与追踪矩阵	126
25	ADR 模板与规范治理流程	132

理解设计

建议阅读 01 → 03 → 05 → 07 → 11 → 13 → 18 → 22。

准备开发

先读 01-12，再读 18-21 与 24，最后回到场景逐项验证。

编写 Agent Skill

重点阅读 02-07、10-17 与黄金样例，禁止只复制文风规则。

PART I

愿景与领域语义

先定义系统中的世界，再讨论如何实现它。

对象不是标签；对象的语义决定它能如何存在、关联、变化与结束。

项目愿景、边界与不可变原则

本章回答

为什么做这套系统，以及哪些原则不可因实现便利而牺牲。

核心结论

确定系统优化的是认知成本，而不是字段、模板或功能数量。

1. 问题陈述

用户的工作、学习和个人项目会从许多入口产生：Journal 中的一句话、会议和聊天中的要求、技术故障、文件和表格、代码仓库、大模型对话、长期构思以及已经形成但尚未归档的成果。它们初始时往往模糊、片段化，且不适合立即被要求填写完整字段。

现有 Logseq 工作区同时承担原始记录、项目说明、任务数据库、工作流、历史档案和当前工作台，造成三类结构性问题：

1. **输入摩擦**：记录新信息时被迫立即决定类型、归属、状态和模板；
2. **阅读污染**：完整属性、历史记录和空结构直接暴露在项目正文中；
3. **重入困难**：项目保存了大量内容，却缺少一份稳定、克制、可行动的当前状态接口。

本项目旨在建立一套个人事务运行系统，使自然语言记录、事务工作流、注意力视图与 Agent 协作能够分层而统一地工作。

2. 核心愿景

系统应当让用户能够放心地记录，因为用户知道：

- 原始信息不会丢失；
- 重要信息可以在之后获得正确语义；
- Agent 会给出透明、可拆分、可修改的处理建议；
- 正式内容经审查后保持可读和简洁；
- 事务会进入合适的工作流；
- 任一项目中断后都可以快速恢复上下文和行动能力。

系统的最终体验不是“维护数据库”，而是：

记录时尽量自由，审查时拥有控制，工作时只看必要信息，回顾时能够看见完整证据。

3. 优化目标

3.1 首要目标

- **降低捕获成本**：用户可以先记录事实和想法，再决定结构。
- **降低重入成本**：中断后能够迅速回答“这是什么、到哪里、为什么、下一步是什么”。
- **降低维护成本**：减少人工维护重复字段、摘要和跨页面副本。
- **降低注意力噪声**：默认视图不展示当前无关的信息。
- **提高内容质量**：正式内容保持自然、完整、重点清晰且不过度模板化。
- **提高系统可信度**：Agent 的每一次正式修改都可见、可审查、可回滚。

3.2 成功迹象

一个可用系统应当表现为：

- 用户随手输入时无需先做完整分类；
- 项目正文比原先更干净，而不是新增更多字段；
- 同一任务在多个视图中出现但不存在多个状态副本；
- 等待、阻塞、停滞和缺少下一步能够被可靠发现；
- Agent 建议接受率逐步提高，而误归属和杜撰率逐步下降；
- 项目重入摘要能在几分钟内恢复行动能力；
- 更换界面或 LLM 时，核心语义与历史仍可保留。

4. 非目标

首个稳定版本明确不以以下能力为目标：

- 多用户组织权限和企业审批；
- 团队级资源排期与工时核算；
- 完整 CRM、财务或采购流程；
- 实时多人协同编辑；
- 取代专业代码仓库、文献管理或文件管理系统；
- 将所有自然语言强制转成结构化字段；
- 自动替用户决定项目目标或重要价值判断；
- 在没有审查的情况下大规模重写历史内容。

这些能力未来可以通过适配或外部集成增加，但不得破坏个人、本地优先、低摩擦和可追溯的核心。

5. 不可变原则

PRI-001

优化认知成本，而不是结构数量

系统 **MUST** 以降低捕获和重入成本为主要目标。新增字段、模板或页面只有在改善理解、推进、检索、恢复或审计时才合法。

PRI-002

自然语言优先

正式正文 **MUST** 以自然、连贯、可直接阅读的语言为主。结构化属性 **MAY** 隐藏存在，但 **MUST NOT** 取代必要的叙事说明。

PRI-003

渐进式正式化

原始输入 **MAY** 不完整、不分类。系统 **MUST** 允许其经过捕获、解释、审查后再进入正式对象和工作流。

PRI-004

正文唯一，历史可追溯

用户批准正式内容后，正式正文 **MUST** 替换原始正文，避免长期并列多个当前版本；原始版本 **MUST** 保存在隐藏历史中，并可被审计和恢复。

PRI-005

用户保持最终控制权

任何影响正式正文、主归属、对象类型、层级、完成、归档、合并或删除的操作 **MUST** 可审查。高影响操作 **MUST** 显式确认。

PRI-006

建议可拆分

Agent 的一次处理 **MUST** 被表示为若干可独立接受、编辑或拒绝的语义操作。文字润色、归属、位置和对对象创建不得捆绑为单一按钮。

PRI-007

位置不等于语义

一个块位于某个页面下 **MUST NOT** 自动等同于其主归属。物理位置服务阅读上下文；主归属、关联和视图由领域关系明确表达。

PRI-008

完整数据与克制视图分离

系统 **MAY** 保存完整结构，但默认视图 **MUST** 只投影当前场景需要的信息。空栏目、无内容结构和内部字段 **MUST NOT** 默认渲染。

PRI-009

单类信息单一权威

每一类信息 **MUST** 有唯一权威来源。Logseq 正文、领域状态、文件产物和生成视图之间不得形成双主竞争。

PRI-010

确定性规则优先

唯一性、状态流转、关系约束、原子提交、版本和回滚 **MUST** 由确定性代码保证。LLM 不得成为这些约束的唯一执行者。

PRI-011

Agent 解释而非伪装确定

Agent **MUST** 区分事实、提取、推断、建议和未知。不得将缺乏证据的推测写成已确认事实。

PRI-012

核心语义独立于前端

Task、MiniProject、Project、Area、关系和生命周期 **MUST** 在独立领域模型中定义。Logseq 是首要适配界面，但不是语义定义本身。

PRI-013

默认可逆

除明确不可逆的外部副作用外，正式修改 **SHOULD** 可撤销。删除 **SHOULD** 先进入软删除或归档阶段。

PRI-014

事件与当前状态分离

发生过什么和现在意味着什么 **MUST** 分开维护。事件历史可以增长，当前状态必须保持克制。

PRI-015

低打扰治理

Agent **SHOULD** 默认在用户主动调用或专门审查时介入。只有高价值异常才可主动提醒，禁止每次输入都强制弹出复杂审批。

6. 设计取舍原则

当两个目标冲突时，按以下优先级处理：

1. 数据与历史不失真；
2. 用户对正式变更的控制权；
3. 捕获和当前工作体验；
4. 跨前端可迁移性；
5. 自动化程度；
6. 功能丰富度。

例如，当自动归属能够减少操作，但误归属会改变项目边界时，应优先保留审查，而不是追求全自动。

7. 项目阶段边界

阶段一：语义原型

验证对象定义、捕获、审查、部分接受、当前工作台和项目重入，不追求完整插件 UI。

阶段二：Logseq 插件 MVP

实现独立任务对象、来源锚点、工作台、抽屉、审查中心和本地存储。

阶段三：Agent 深度协作

增加项目摘要、停滞分析、迁移建议、批量审查和受控 MCP/API。

阶段四：跨前端与外部资源

增加文件工作区、CLI/Web 前端、Zotero/Git 集成和更健壮同步。

8. 规范冲突处理

若下游文档与本文件冲突，必须以本文件为准，并通过 ADR 解决。任何违反不可变原则的实现不能仅以“开发方便”“现有 API 限制”作为长期理由；只能作为有期限的技术债记录。

核心概念与术语表

本章回答	核心结论
统一 Task、MiniProject、Project、Area、Capture 等核心术语。	消除同一个词在正文、Agent、数据模型和界面中的语义漂移。

1. 术语使用规则

本文件是术语的唯一权威定义。其他文档 **MUST** 引用这里的定义，不得为同一词重新赋予不同含义。界面可以使用更自然的中文显示名，但内部名称必须保持一致。

2. 核心总称

事务对象 (Managed Object)

系统中拥有稳定 ID、可被引用、具有明确语义并可接受领域操作的对象总称。不是所有块、段落和属性都是事务对象。

工作对象 (Work Object)

需要被推进、维护或结束的事务对象，包括 Task、MiniProject、Project 和 Area。Area 虽无天然完成时刻，但仍需要周期维护。

支撑对象 (Supporting Object)

用于解释、证明、关联或沉淀工作的对象，包括 Capture、Decision、Output、Resource、ExternalArtifact 和 PersonRef。

系统对象 (System Object)

用于实现审查、提交、历史和视图的内部对象，包括 Proposal、SemanticOperation、SemanticCommit、DomainEvent、ViewDefinition 和 Anchor。

3. 工作对象

Task

可以被明确承诺、直接执行，并能以清晰条件判断完成与否的最小工作对象。Task 可以有背景和完成标准，但不应维护复杂的独立治理结构。

MiniProject

为实现一个有限且相对独立的结果，需要多个步骤、判断或局部上下文的工作对象。它是 Project 可管理的整体，同时拥有自己的进展和子任务。

Project

拥有独立目标、边界、成果和生命周期，需要协调多个工作单元、决策或阶段的工作对象。Project 的结束应产生可判断的目标结果或明确取消结论。

Area

长期存在、没有天然完成时刻的责任范围。Area 通过维持标准、履行职责和容纳相关 Project 来体现价值；它不是“大 Project”。

4. 支撑对象

Capture

尚未正式化的原始输入，可以是一句话、片段、文件、聊天内容、语音转写或想法。Capture 不等于 Task，也不要立即拥有正式归属。

Context Note / 正文锚点

在 Logseq 或其他知识工作区中保存自然语言上下文的内容位置。它可以是对象的主要正文，也可以只是来源或补充背景。

Decision

对选项、约束或方向作出的具有后续影响的明确判断。Decision 必须记录结论、依据、状态和适用范围；普通想法不自动成为 Decision。

Output

工作过程中形成的可交付、可复用或可引用成果，例如报告、脚本、方案、表格、演示、经验文档。Output 不等于“已完成任务”，它是工作产生的资产。

Resource

为理解或执行工作提供参考的信息集合，例如资料页、教程、规范和阅读笔记。Resource 通常没有明确完成结果。

ExternalArtifact

存放于 Logseq 之外的文件、仓库、文献或外部对象，例如本地目录、Git 仓库、Zotero 条目、PDF、压缩包或在线文档。

PersonRef

对相关人员或角色的引用。PersonRef 用于表达负责人、等待对象、咨询对象和影响对象，但系统首版不承担完整联系人管理。

5. 系统对象

Anchor

领域对象与外部内容位置之间的稳定映射。Anchor 包含对象 ID、适配器类型、外部标识和角色，例如主正文、来源、上下文或产物。

Proposal

Agent 对一次输入或一组对象提出的待审查建议集合。Proposal 本身不是正式事实。

SemanticOperation

Proposal 中可独立接受、编辑或拒绝的一项语义变化，例如改写正文、改变主归属、创建任务或更新状态。

SemanticCommit

用户确认后，以原子方式提交的一组 SemanticOperation。SemanticCommit 具有唯一 ID、版本前置条件、执行结果和可撤销信息。

DomainEvent

系统中已经发生的事实记录，例如对象创建、状态变化、正文替换、关系增加或提交撤销。DomainEvent 是审计历史的一部分。

ViewDefinition

对领域数据的查询、排序、投影和显示规则。ViewDefinition 不保存任务副本，也不成为状态权威。

6. 关系术语

主归属 (Primary Ownership)

对象在责任和主要上下文上的唯一直接父级关系。一个工作对象在同一时刻最多有一个主归属。

关联 (Association)

不改变主归属的跨对象关系，例如相关主题、共同人员或相互参考。关联可以多对多。

物理位置 (Placement)

内容在 Logseq 页面和块树中的实际位置。物理位置用于阅读组织，不自动决定主归属。

来源 (Source)

信息或对象从哪里产生。来源保留证据和时间上下文，不因内容移动而丢失。

上下文 (Context)

理解或执行对象时有帮助的内容，但不一定是其来源或主归属。

依赖 (Dependency)

对象完成或推进需要另一个对象先满足某个条件。依赖关系有方向，并禁止循环。

阻塞 (Blocking)

依赖尚未满足而导致当前不可推进的具体状态。`A blocks B` 是 `B depends_on A` 的反向表达。

产出 (Produces)

工作对象形成 Output 或 ExternalArtifact 的关系。

7. 状态术语

生命周期阶段 (Lifecycle Phase)

对象从出现到结束所处的主要阶段，例如 Clarify、Ready、Active、Closing、Completed。阶段表达“走到了哪里”。

执行条件 (Execution Condition)

对象当前是否可推进，例如 Actionable、Waiting、Blocked、Paused。执行条件表达“现在能不能推进及为什么”。

注意力信号 (Attention Signal)

根据事实动态计算的提示，例如 Overdue、ReviewDue、Stale、NoNextAction。注意力信号不是永久手填状态。

Waiting

当前推进责任暂时位于外部主体或事件，系统知道正在等什么以及何时复查。

Blocked

当前无法推进，且阻碍条件未被有效处理或责任不明确。Blocked 不等于 Waiting。

Paused

用户主动决定暂时不投入注意力，但对象仍然有效。

Stale

由时间和活动规则计算的“可能停滞”信号，不是用户直接设置的生命周期状态。

8. 内容术语

原始内容

用户首次输入、尚未经正式审查的内容。

正式内容

经用户审查确认后，作为当前有效正文使用的内容。正式内容可以由用户直接写成，也可以来自 Agent 建议。

当前状态摘要

面向当前理解和行动的短叙事，它是可再生或可人工维护的投影，不替代事件历史。

事件流

按时间保存“发生了什么”的记录。

重入包

为中断后恢复项目而生成的视图，包含目标、当前状态、最近变化、关键结论、未决问题、必要资料和建议恢复动作。

9. 操作术语

捕获 (Capture)

将输入安全地纳入系统，而不要求立即完成语义分类。

分诊 (Triage)

判断 Capture 是新对象、现有对象进展、资料、重复项、无行动信息或待确认内容。

正式化 (Formalization)

将 Capture 经过语义解释和审查转化为正式正文、对象、关系或事件。

语义提交 (Semantic Commit)

把用户确认的语义操作原子写入各自权威来源。

投影 (Projection)

从完整领域数据中选择适合某一注意力场景的信息进行展示。

重入 (Re-entry)

从中断状态恢复对事务的理解和行动能力。

事务对象语义与契约规范

本章回答

定义一等事务对象的最低契约、适用边界与反例。

核心结论

对象类型由行为和生命周期决定，而不是由文本长度决定。

事务对象的区别不在“大小”，而在治理行为与生命周期



图 4 Task、MiniProject、Project 与 Area 由治理语义区分，不是简单的尺寸梯度。

1. 目的

本规范定义系统中的一等对象及其必须满足的语义契约。对象类型不是装饰标签，而是决定其生命周期、关系、视图、操作和 Agent 权限的领域类型。

2. 通用对象契约

所有正式事务对象 **MUST** 具备：

- 稳定且不可复用的 `object_id` ；
- 明确 `object_type` ；
- 创建时间和最近修改时间；
- 当前生命周期阶段；
- 至少一个可追溯来源或创建事件；
- 可选但稳定的主正文锚点；

- 版本号，用于并发和提交前置检查。

所有工作对象 **SHOULD** 具备：

- 一段面向人的简洁说明；
- 明确的当前状态解释；
- 适合该粒度的下一步或维护动作；
- 主归属或明确的“待确认归属”状态。

SEM-COMMON-001

对象类型 **MUST** 由语义行为决定，不得只按文本长度、子块数量或标题形式判断。

SEM-COMMON-002

不存在有效内容的可选结构 **MUST NOT** 为了模板完整而生成。

SEM-COMMON-003

正文内容不足以满足对象最低契约时，对象 **MUST** 处于 Clarify/Defining，而不得伪装为 Ready/Active。

3. Task 契约

3.1 存在目的

Task 表达一个可以被直接承诺和执行的工作单元。

3.2 必要条件

一个内容成为 Task，至少满足：

1. 存在可理解的动作或预期变化；
2. 完成条件可以被判断；
3. 当前粒度不需要独立维护复杂的多个工作流；
4. 能够由一个主要责任主体推进。

3.3 必须暴露的信息

Ready 及之后的 Task **MUST** 暴露：

- 标题或自然语言动作；
- 完成标准；
- 当前阶段和执行条件；

- 主归属；
- 下一可执行动作，若标题本身已是原子动作可与标题相同。

3.4 可选信息

- 截止时间；
- 复查时间；
- 等待对象；
- 依赖；
- 必要背景；
- 相关文件或来源；
- 预估投入和执行环境。

3.5 禁止承担

Task **MUST NOT** 承担：

- 多个彼此独立的目标；
- 长期责任标准；
- 大量需要持续维护的决策、里程碑和成果集合；
- 无明确行动意图的纯资料。

SEM-TASK-001

Task **MUST** 具有可判断的完成结果。“研究一下”“持续关注”若没有明确终态，不能直接成为 Ready Task。

SEM-TASK-002

Task 的背景 **SHOULD** 足以避免未来重新理解，但 **MUST NOT** 复制完整项目上下文。

3.6 正例与反例

正例：

使用一条非 Test 真实事件，验证 Graylog 从 Event Definition 到外部推送的完整链路，并保存验证证据。

反例：

完善 Graylog。

后者缺少边界、动作和完成标准，应继续 Clarify 或升级为更高粒度对象。

4. MiniProject 契约

4.1 存在目的

MiniProject 表达一个有限结果：它不能由单一动作完成，但又不需要成为独立 Project 的治理中心。

4.2 必要条件

一个内容成为 MiniProject，通常同时满足：

- 有明确且有限的目标结果；
- 需要两个以上具有顺序、判断或协作关系的工作步骤；
- 需要维护局部上下文或进展；
- 能够作为整体被上层 Project 或 Area 管理；
- 完成后不需要继续长期维持独立责任。

4.3 必须暴露的信息

Ready 及之后的 MiniProject **MUST** 暴露：

- 目标结果；
- 边界；
- 当前状态摘要；
- 当前推进或下一阶段；
- 子任务或内部推进结构；
- 主归属；
- 完成判据。

4.4 可选信息

- 关键决策；
- 局部风险；
- 里程碑；
- 产出；
- 参与人；
- 时间范围；
- 文件工作区。

4.5 禁止承担

MiniProject **SHOULD NOT** :

- 长期无终点地持续存在；
- 同时拥有多个独立目标和多个上层利益相关方；
- 成为大量 MiniProject 的长期父级；
- 需要独立项目级治理却仍以“Mini”为名保留。

SEM-MINI-001

MiniProject 的“Mini”由治理独立性和边界决定，不由任务数量决定。

SEM-MINI-002

一个只有三行内容但拥有独立目标和边界的工作，可能是 Project；一个拥有十个简单步骤但完全服务于单一有限结果的工作，仍可能是 MiniProject。

5. Project 契约

5.1 存在目的

Project 负责交付一个相对独立的目标结果，并维护实现该结果所需的边界、阶段、决策、工作单元和成果。

5.2 必要条件

Project 至少满足：

- 存在可描述的目标状态；
- 存在明确的包含与不包含边界；
- 需要协调多个工作单元、阶段或关键判断；
- 拥有相对独立的成果或结束结论；
- 需要在一段时间内持续维护当前状态。

5.3 必须暴露的信息

Planned/Active Project **MUST** 暴露：

- 项目目的与目标结果；
- 范围边界；
- 当前阶段；
- 当前状态摘要；
- 最近重要变化；

- 当前推进和待决策；
- 阻塞、等待或暂停原因；
- 关键成果入口；
- 下层对象入口；
- 完成判据。

5.4 可选信息

- 利益相关人；
- 里程碑；
- 风险；
- 假设；
- 预算或资源；
- 文件工作区；
- 复盘和经验；
- 外部系统链接。

5.5 禁止承担

Project **MUST NOT**：

- 仅因一个主题值得关注就建立，而没有目标结果；
- 无限容纳所有相关资料；
- 以“持续优化”“长期学习”作为无边界目标；
- 用项目页直接展示全部历史和字段作为默认工作界面。

SEM-PROJ-001

Project 的核心不是任务集合，而是目标、边界、阶段和成果的独立治理单元。

SEM-PROJ-002

Project 若长期无法定义结束条件，系统 **SHOULD** 建议其转为 Area、Resource 或拆分为可结束 Project。

6. Area 契约

6.1 存在目的

Area 表达需要长期维持的责任、能力或生活领域。

6.2 必要条件

- 没有天然完成时刻；
- 存在持续责任或维持标准；
- 可以产生多个独立 Project；
- 价值体现在长期状态，而非一次交付。

6.3 必须暴露的信息

Active Area **MUST** 暴露：

- 责任范围；
- 维持标准或期望状态；
- 当前活跃 Project；
- 需要周期关注的指标或风险；
- 明确的边界和非职责范围。

6.4 禁止承担

Area **MUST NOT** 使用 Done 作为正常终态；不得用大量零散 Task 替代 Project 规划。

SEM-AREA-001

“学习 Linux” 若表示持续能力建设，属于 Area；“通过 2026 年 RHCSA 考试” 属于 Project。

7. Capture 契约

Capture 允许：

- 不完整；
- 无正式归属；
- 含推测或口语；
- 与其他内容重复；
- 暂时无法判断是否需要行动。

Capture **MUST** 具有来源时间和原始内容。Capture 在正式化后可以被正文替换，但原始版本必须进入历史。

SEM-CAP-001

Capture 只是输入状态，不得被默认当作 Task。

8. Decision 契约

Decision 必须包括：

- 决策问题；
- 最终结论；
- 主要依据；
- 适用范围；
- 决策状态：proposed/accepted/superseded/revoked；
- 相关对象。

SEM-DEC-001

Agent 推断不得直接创建 accepted Decision。只有用户明确确认或已有权威证据时才能接受。

9. Output 契约

Output 必须说明：

- 产物是什么；
- 来源工作对象；
- 当前版本或位置；
- 是否为最终交付、草稿或中间证据；
- 后续可复用价值。

完成 Task 不自动生成 Output；只有存在可独立引用的成果时才创建。

10. Resource 与 ExternalArtifact 契约

Resource 用于知识意义上的组织；ExternalArtifact 用于对外部实际对象的引用。一个 PDF 可以同时是 ExternalArtifact，并被某个 Resource 页面组织。

SEM-RES-001

Resource 不应被强制纳入执行状态机。若阅读资源需要行动，应另建 Task 并关联 Resource。

11. 对外接口的分层

每个工作对象可以拥有完整内部数据，但 **MUST** 提供以下三个接口：

1. **Compact Interface**：标题、阶段、执行条件、下一步；
2. **Working Interface**：目标/完成标准、当前状态、必要背景、阻塞和操作；
3. **Re-entry Interface**：目的、边界、最近变化、关键结论、未决问题、恢复动作、资料入口。

不同视图只能投影接口，不得复制并自行维护一套状态。

关系、归属与层级规范

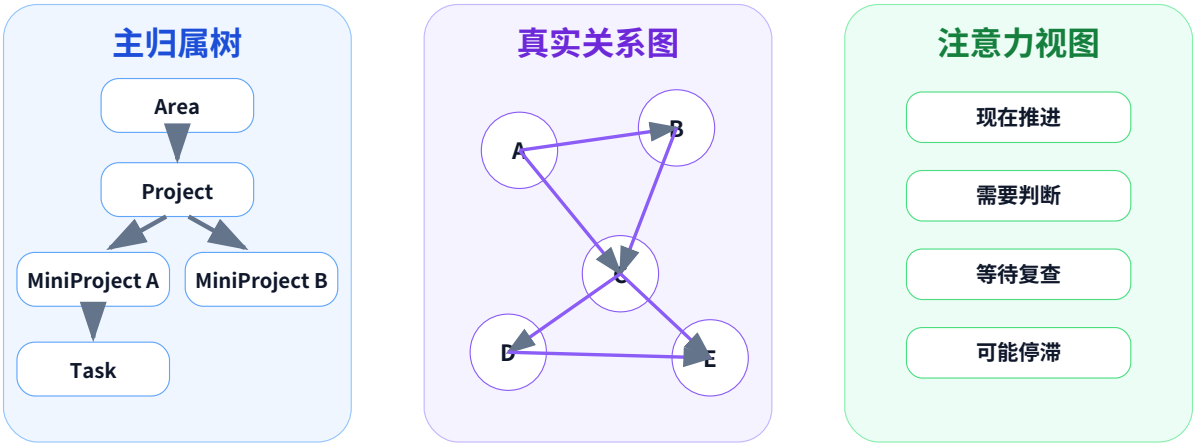
本章回答

区分主归属、物理位置、关联、来源、上下文和依赖。

核心结论

树负责稳定归属，图负责真实关系，视图负责当下聚合。

树、图与视图各自解决不同问题



稳定归属不妨碍跨项目聚合；跨项目聚合也不应改变对象本身。

图 5 主归属树、真实关系图和注意力视图相互配合。

1. 设计目标

本规范将“内容放在哪里”和“它属于什么”解耦，并建立稳定树状主归属与跨树关系共存的模型。

2. 关系分类

2.1 结构关系

- `owned_by`：主归属，单向；
- `contains`：主归属的反向投影；
- `parent_task`：Task 的执行分解关系；
- `parent_work`：MiniProject/Project 的层级关系。

2.2 执行关系

- `depends_on` ；
- `blocks` ；
- `precedes` ；
- `duplicates` ；
- `supersedes` 。

2.3 证据和上下文关系

- `sourced_from` ；
- `contextualized_by` ；
- `documented_at` ；
- `decided_by` ；
- `produces` ；
- `references_artifact` ；
- `involves_person` 。

2.4 弱关联

- `related_to` ：只有在无法用更具体关系表达时使用。

3. 主归属规则

REL-OWN-001

一个工作对象在同一时间最多拥有一个直接主归属。

REL-OWN-002

主归属表达责任和主要解释上下文，而不是物理存放位置。

REL-OWN-003

Task 可以直接归属于 MiniProject、Project 或 Area。系统不得为了形式完整而强制建立空 MiniProject。

REL-OWN-004

MiniProject 通常归属于 Project，也可以直接归属于 Area；若直接归 Area，应说明为什么无需独立 Project。

REL-OWN-005

Project 通常归属于 Area，也可以暂时处于 Unassigned/Incubating 区域，但 Active Project **SHOULD** 明确责任 Area。

REL-OWN-006

Area 不得相互形成无限层级。首版最多允许两层 Area，且子 Area 必须有清楚的责任边界。

4. 树与图的分工

主归属形成一棵或若干棵稳定树，用于回答：

- 谁负责这件事；
- 上层为何关心它；
- 默认从哪里进入它的上下文；
- 项目结束时如何收束下层工作。

跨项目视图和关联形成图，用于回答：

- 哪些事项都在等待同一个人；
- 哪些项目引用同一份文件；
- 哪些任务都与 Graylog 有关；
- 哪些对象相互依赖。

REL-GRAPH-001

视图聚合不得改变主归属。

REL-GRAPH-002

对象可出现在任意数量的视图中，但状态和内容不得复制为多个可独立编辑副本。

5. 物理位置规则

REL-PLC-001

Logseq 中正文块可以留在 Journal、项目页或主题页，位置本身不构成主归属。

REL-PLC-002

移动块是独立语义操作，必须与文字改写和归属修改分别审查。

REL-PLC-003

若内容留在 Journal，但已正式关联 Project，Project 视图应通过引用或投影显示必要信息，不得要求复制原文。

REL-PLC-004

物理移动不得破坏来源时间和原始上下文。系统必须保留 `sourced_from` 锚点。

6. 依赖关系规则

REL-DEP-001

`depends_on` 必须具有方向，且领域层禁止循环依赖。

REL-DEP-002

只有当前对象确实不能达到某个阶段或完成条件时才建立依赖；普通相关性不得伪装为依赖。

REL-DEP-003

Waiting 不一定需要依赖对象，但必须记录等待主体/事件、期待结果和复查时间。

REL-DEP-004

Blocked **SHOULD** 尽量关联具体 blocker；若未知，必须记录阻塞说明并进入待澄清。

7. 来源与上下文

REL-SRC-001

每个正式化自 Capture 的对象或正文变更必须保留来源锚点。

REL-SRC-002

来源表达“从哪里来”；上下文表达“理解时有帮助”。两者不得混用。

REL-SRC-003

同一个来源可以产生多个对象；同一个对象也可以有多个来源。

8. 人员关系

人员关系至少区分：

- **owner**：主要推进责任；
- **waiting_for**：当前等待对象；
- **consult**：需要咨询；
- **stakeholder**：受结果影响；
- **mentioned**：普通提及。

首版个人系统默认 owner 为用户本人，只有明确需要时才显示。

9. 产出关系

Project、MiniProject 和 Task 均可产生 Output。Output 归档后仍保留与产生它的工作对象的关系。

REL-OUT-001

Project 完成前 **SHOULD** 检查是否存在已经形成但未登记的 Output。

10. 冲突与优先级

当物理位置、文本表述和领域主归属不一致时：

1. 领域模型中的已确认主归属是权威；
2. Logseq 页面标题和结构仅作为提示；
3. 系统应生成“映射不一致”审查建议，不得静默重写；
4. 用户可选择修改归属、移动内容或保留现状。

11. 边界案例

案例一：一条 Journal 记录同时涉及两个项目

主归属选择主要责任 Project；另一个建立 `related_to` 或更具体关系。原文可以继续留在 Journal。

案例二：一个 MiniProject 被多个 Project 使用

若它真正产生共享能力，应升级为独立 Project 或归属于公共 Area；不得拥有两个主归属。

案例三：同一任务为两个项目服务

优先识别任务是否包含两个完成结果并拆分。若不可拆分，选择承担主要结果的 Project 为主归属，另一个作为受益关联。

生命周期与状态机规范

本章回答

把生命周期、执行条件和注意力信号分离。

核心结论

避免 Doing、Waiting、Blocked、Stale 混成一条不断膨胀的状态轴。

三轴状态模型：把“走到哪里”“能否推进”“为何提醒”分开



示例：一个 Task 可以同时是 ACTIVE + WAITING，并因 review_at 到期产生 REVIEW_DUE。

图 6 三轴模型避免状态爆炸，并使提醒可以重新计算。

1. 两轴状态模型

为避免 Doing / Waiting / Blocked / Paused / Done 混在一条状态轴上造成状态爆炸，系统将状态拆为：

- 1. **生命周期阶段 Phase**：对象从出现到结束走到哪里；
- 2. **执行条件 Condition**：当前能否推进及为什么；
- 3. **注意力信号 Signal**：由事实计算出的提醒，不作为人工状态。

LIF-BASE-001

Phase、Condition 和 Signal **MUST** 独立存储和计算，不得互相代替。

2. 通用执行条件

- **ACTIONABLE**：当前存在可执行动作；
- **WAITING**：外部主体或事件正在承担下一变化；
- **BLOCKED**：无法推进且阻碍尚未有效处理；
- **PAUSED**：用户主动暂停投入；
- **NONE**：已完成、取消、归档或尚不适用。

LIF-COND-001

WAITING 必须具有 **waiting_for**、期待结果和 **review_at**。

LIF-COND-002

BLOCKED 必须具有阻塞说明；**SHOULD** 关联 blocker 或待澄清动作。

LIF-COND-003

PAUSED 必须记录暂停理由和可选复查时间。

3. 注意力信号

由系统动态计算：

- **OVERDUE**：截止时间已过且未结束；
- **REVIEW_DUE**：复查时间已到；
- **STALE**：活跃对象超过阈值没有有效事件；
- **NO_NEXT_ACTION**：Ready/Active 对象缺少下一动作；
- **UNASSIGNED**：应有主归属但尚未确认；
- **BOUNDARY_DRIFT**：范围疑似扩大；
- **UNHARVESTED_OUTPUT**：完成工作有产物但未登记；
- **CONFLICT**：映射或版本冲突。

LIF-SIG-001

Signal **MUST** 可根据事实重新计算，不得由用户手工当作永久标签维护。

4. Capture 生命周期

Phase:

NEW → PROPOSED → RESOLVED

分支:

- NEW/PROPOSED → DISMISSED
- PROPOSED → NEW (撤回建议)

含义:

- NEW: 尚未分析或仅保存;
- PROPOSED: 已生成 Agent 建议;
- RESOLVED: 已正式化、合并进现有对象或确认无需行动;
- DISMISSED: 用户拒绝处理且不作为正式信息保留。

5. Task 生命周期

Phase:

CLARIFY → READY → ACTIVE → COMPLETED

可选分支:

- CLARIFY/READY/ACTIVE → CANCELLED
- COMPLETED/CANCELLED → ARCHIVED
- COMPLETED → ACTIVE (重新打开, 必须记录原因)

含义:

- CLARIFY: 动作、完成标准或归属不足;
- READY: 已明确且可以进入执行队列;
- ACTIVE: 已承诺推进或正在执行;
- COMPLETED: 完成标准已满足;
- CANCELLED: 明确不再需要;
- ARCHIVED: 退出日常工作视图但保留历史。

LIF-TASK-001

Task 进入 READY 前必须满足 SEM-TASK-001。

LIF-TASK-002

Condition WAITING/BLOCKED/PAUSED 可与 READY 或 ACTIVE 组合，而不是取代 Phase。

LIF-TASK-003

Task 完成必须记录完成事件；若存在 Output，应建立 produces 关系。

6. MiniProject 生命周期

DEFINING → READY → ACTIVE → CLOSING → COMPLETED → ARCHIVED

分支：

- 任意未结束阶段 → CANCELLED；
- ACTIVE → READY（尚未真正开始或重新规划）；
- COMPLETED → ACTIVE（重新打开，需高影响确认）。

进入 READY 的门槛：目标、边界、完成判据、主归属和至少一个当前推进方向明确。

进入 CLOSING 的门槛：主要目标已完成，剩余工作主要是验证、交付、总结或归档。

7. Project 生命周期

IDEA → DEFINING → PLANNED → ACTIVE → CLOSING → COMPLETED → ARCHIVED

分支：

- IDEA/DEFINING/PLANNED/ACTIVE → CANCELLED；
- ACTIVE 可配合 PAUSED/WAITING/BLOCKED 条件；
- COMPLETED 重新打开需要新 SemanticCommit 和理由；
- IDEA 可以长期存在，但不进入活跃注意力视图。

LIF-PROJ-001

Project 进入 PLANNED 前必须具有目标、边界、完成判据和责任 Area 或明确例外。

LIF-PROJ-002

Project 进入 ACTIVE 前必须存在当前推进或可执行下层对象。

LIF-PROJ-003

Project 进入 COMPLETED 前必须检查：目标结果、未完成下层对象、未决等待、成果登记、归档入口和复盘需要。

8. Area 生命周期

ACTIVE ↔ DORMANT → RETIRED

- ACTIVE：持续承担责任；
- DORMANT：当前不投入，但未来可能恢复；
- RETIRED：责任不再存在，相关项目和资源已迁移或归档。

Area 无 COMPLETED。

9. 状态转换权限

低影响，可由规则自动执行

- 根据日期增加/移除注意力信号；
- 由合法事实将 Capture 标记 RESOLVED；
- 在所有依赖完成后移除某些 BLOCKED 提示，但不得自动切换用户主动设置的 PAUSED。

中影响，可由 Agent 建议，通常需确认

- Task CLARIFY → READY；
- READY → ACTIVE；
- 设置 WAITING/BLOCKED；
- 重新打开已完成 Task。

高影响，必须显式确认

- Project/MiniProject COMPLETED、CANCELLED、ARCHIVED；
- Area RETIRED；
- 已结束对象重新打开；
- 伴随对象类型迁移的状态变化。

10. 合法性校验

每次状态转换 **MUST** 校验：

- 当前版本未变化；

- 转换在允许矩阵内；
- 进入门槛满足；
- 必要字段存在；
- 相关依赖和下层对象处理符合规则；
- 操作者权限足够；
- 写入审计事件。

11. 典型误用

- 用 High Priority 代替 Active：错误，优先级和生命周期不同；
- 用 Waiting 代替截止日期：错误，Waiting 必须指外部等待；
- 把长期未动自动标为 Cancelled：禁止，系统只能发出 STALE 信号；
- 把 Project 标 Done 但仍有大量活跃子任务：必须先处理或明确迁移。

对象迁移、拆分与合并规范

本章回答

规定对象何时升级、降级、拆分、合并或改变主归属。

核心结论

迁移是语义变化，不等同于移动 Logseq 块。

1. 目的

对象粒度不是一经创建便永久不变。随着目标、范围和治理复杂度变化，Task 可能升级为 MiniProject，MiniProject 可能升级为 Project，Project 也可能被拆分、合并、降级或转为 Area。迁移必须依据语义变化，而不是依据文本长度或主观感觉。

2. 通用原则

MIG-001

身份连续性

迁移 **SHOULD** 保留原对象 ID，并通过 `type_changed` 事件记录前后类型；只有拆分为多个独立对象时才创建新 ID。

MIG-002

历史不可重写

迁移不得伪装成“它从一开始就是新类型”。历史视图必须能看到原类型、迁移时间、依据和用户确认。

MIG-003

建议与执行分离

Agent **MAY** 建议迁移，但不得自动执行 Task↔MiniProject、MiniProject↔Project、Project↔Area 的类型变化。

MIG-004

迁移不等于移动

对象类型迁移、主归属变化和 Logseq 物理位置移动是三项独立操作，必须允许分别接受。

3. Task 升级为 MiniProject

至少出现两个强信号时可建议升级：

- 单一动作已演变为多个需要独立跟踪的步骤；
- 需要维护局部决策、验证路径或多次进展；

- 完成标准包含多个非平凡条件；
- 当前 Task 标题无法代表内部工作的整体目标；
- 为保持可读性，已经不得不在 Task 下建立复杂结构。

不得仅因为：

- 有多个简单检查项；
- 文本较长；
- 执行时间超过一天；
- Agent 可以想象出更多步骤。

迁移时必须补充 MiniProject 目标、边界、完成判据和当前推进。

4. MiniProject 升级为 Project

建议升级需要证明其治理独立性增强，典型依据：

1. 形成独立目标，不再只是上层 Project 的一个有限结果；
2. 出现稳定边界和明确的非范围；
3. 拥有多个 MiniProject、阶段或关键决策；
4. 产生独立成果和利益相关人；
5. 上层 Project 即使结束，该工作仍需继续；
6. 继续留在上层会显著增加其理解和维护负担。

MIG-UP-001

“内容很多”不能作为唯一升级依据。

MIG-UP-002

升级为 Project 后必须重新确定责任 Area、目标终态和完成判据。

5. Project 转为 Area

当一个 Project 长期无法定义结束，并且实际承担的是持续维持责任时，可建议转为 Area。例如“持续维护个人知识系统”更像 Area，而“完成个人知识系统 v1”是 Project。

转为 Area 前必须处理：

- 将可结束目标拆为 Project；
- 定义长期维持标准；
- 迁移已有下层对象；

- 移除 Completed 终态预期。

6. 降级

Project 降为 MiniProject

适用于：目标实际非常有限、治理边界依赖上层 Project、无需独立成果和长期状态维护。

MiniProject 降为 Task

适用于：经澄清后只剩一个可直接执行和判断完成的动作。

降级不得删除原有决策、事件或成果关系；不再适用的结构进入历史或隐藏。

7. 拆分

出现下列情况时应考虑拆分：

- 一个对象包含多个可以独立完成或取消的目标；
- 不同部分拥有不同主归属；
- 不同部分处于完全不同生命周期；
- 一个 Task 实际上包含“并且”连接的多个独立承诺；
- Project 范围持续膨胀且无法用一个完成判据约束。

拆分操作必须：

1. 创建新对象；
2. 明确每个对象的目标、归属和状态；
3. 将来源、上下文、子对象和产物按语义分配；
4. 原对象标记为 `superseded_by` 或保留为容器；
5. 记录未能自动分配的关系。

8. 合并与重复

只有满足以下条件才允许合并：

- 实际目标相同或一个完全包含另一个；
- 完成判据兼容；
- 主归属可统一；
- 历史和来源可无损保留。

合并时选择一个 canonical object；其他对象进入 `merged` / `superseded` 状态并建立 `superseded_by`。禁止物理删除来源。

MIG-MERGE-001

相同主题不等于重复目标。两个都与 Graylog 有关的 Project 不应因此合并。

9. 主归属迁移

改变主归属必须说明：

- 原归属为何不再合适；
- 新归属提供什么主要责任上下文；
- 对上层项目状态和成果是否有影响；
- 是否需要移动正文。

归属迁移不自动移动 Logseq 内容。

10. 迁移建议格式

Agent 的迁移建议 **MUST** 包含：

- 当前类型与建议类型；
- 命中的语义规则；
- 支持证据；
- 反对迁移的信号；
- 迁移后的目标、边界、归属和状态草案；
- 受影响的关系、视图和正文位置；
- 可独立审查的操作列表。

PART II

信息、提交与审查

让模糊输入安全地成为正式内容，同时不牺牲用户控制权。

正文可以被替换，但历史不能被抹除；建议可以被接受，但必须能够拆分。

信息分层与权威来源规范

本章回答

明确正文、领域状态、审计历史、生成视图和外部产物的权威来源。

核心结论

不是一个工具保存全部真相，而是每类信息只有一个权威。

单类信息单一权威：每一层只负责自己最擅长的真相

Logseq 正文	自然语言、项目说明、决策理由
领域模型	对象身份、状态、主归属、关系
审计事件	原始内容、变更、提交与回滚
文件 / Git	脚本、配置、表格、代码与版本
Zotero 等	文献元数据、PDF 与引用
生成视图	工作台、重入包、聚合摘要

视图可以重新生成；正文与领域状态发生冲突时，必须显式提示，禁止静默覆盖。

图 7 按信息类型划分权威，避免 Logseq 与独立模型形成双主。

1. 核心问题

系统同时包含自然语言、结构化状态、历史事件、生成摘要和外部文件。若它们都可以相互覆盖，就会形成多个“真相来源”。本规范按信息类型分配权威，而不是要求一个工具保存全部内容。

2. 信息层

2.1 捕获层

保存刚进入系统的原始输入和来源。权威是 Capture 记录及其原始版本。

2.2 正文层

保存面向人的正式自然语言内容，包括项目说明、进展、决策理由和上下文。默认权威是 Logseq 中的指定正文锚点。

2.3 领域状态层

保存对象 ID、类型、Phase、Condition、主归属、关系、日期和版本。权威是独立 Domain Store。

2.4 事件与审计层

保存已经发生的修改、提交、状态变化和原始版本。权威是 append-only Audit/Event Store。

2.5 视图层

保存查询和投影定义；视图内容由其他层生成，不是事实权威。

2.6 外部产物层

保存代码、文件、文献和大型附件。权威是文件系统、Git、Zotero 或对应外部系统。

3. 权威矩阵

信息	权威	可缓存位置	禁止行为
正文当前版本	Logseq 主正文锚点	搜索索引、摘要缓存	数据库隐藏副本反向覆盖正文
对象类型和状态	Domain Store	Logseq UI 投影	从页面位置猜测后静默覆盖
主归属和依赖	Domain Store	页面链接、查询结果	在多个页面复制独立可编辑关系
原始输入和旧版本	Audit Store	Git 历史	审查后彻底删除全部证据
当前摘要	可再生生成投影，或指定人工正文	缓存	当成唯一历史记录
文件内容	外部权威系统	元数据索引	把大文件内容复制进字段
Agent Proposal	Proposal Store	UI 临时状态	未确认即视作正式事实

4. 单一权威规则

INF-OWN-001

每个字段或内容片段必须声明 authority class。系统不得允许两个位置同时作为可写权威。

INF-OWN-002

Logseq 正文变化可触发领域建议，但不得仅凭自然语言自动改变高影响领域状态。

INF-OWN-003

Domain Store 可以把状态投影为徽标、悬浮窗或查询块，但这些投影不得成为独立编辑副本。

INF-OWN-004

生成视图可随时删除和重建。任何无法重建的关键事实不得只存在于视图缓存。

5. 正文与结构的协同

正文可以表达：“当前已经完成静态关系梳理，下一步需要真实事件验证。”

领域层则存储：

- phase = ACTIVE;
- condition = ACTIONABLE;
- next_action_id;
- related_output;
- updated_at。

二者语义不一致时，不应自动选择一方覆盖另一方，而应产生 **CONFLICT** Proposal，由用户选择：

- 更新领域状态；
- 修改正文；
- 声明正文只是一段历史描述；
- 延后处理。

6. 事件流与当前状态

INF-EVT-001

过程性进展以事件形式持续增长；当前摘要只表达仍然有效的结论和状态。

INF-EVT-002

Agent 更新当前摘要时必须引用导致变化的事件，不得凭空重写。

INF-EVT-003

旧判断被推翻时，应在当前摘要中移除或标记失效，但历史事件不得删除。

7. 属性最小暴露

属性可以存储于 Domain Store，必要的锚点标识可以存于 Logseq 元数据，但项目正文 **MUST NOT** 为了机器解析而暴露大量 `key:: value`。

Logseq 中只允许以下最小机器标识默认存在：

- 系统管理的对象/锚点 ID；
- 必要的版本或适配器元数据；
- 用户明确希望作为可读内容展示的少量属性。

8. 可移植性

Domain Store 必须支持开放导出：

- JSON/JSONL：对象、关系、事件、提交；
- Markdown：人类可读对象摘要；
- CSV：任务和关系的分析导出；
- manifest：外部文件和锚点。

Logseq 失效时，用户仍应能重建对象结构和阅读正文；Domain Store 失效时，正文和文件仍应可访问。

Logseq 映射与对象锚点规范

本章回答

设计领域对象与 Logseq Block/Page 之间稳定、可恢复的锚点。

核心结论

块可以移动和改写，对象身份不能随物理位置漂移。

1. 映射目标

Logseq 是主要输入和自然语言工作区，但领域对象不能依赖页面路径或块位置获得身份。映射层必须允许正文移动、替换、拆分和重组，同时保持对象关系稳定。

2. Anchor 模型

Anchor 至少包含：

- `anchor_id` ；
- `object_id` ；
- `adapter = logseq` ；
- `graph_id` ；
- `external_id` ： 优先 Block UUID；
- `role` ： `primary_text/source/context/event/output/index`；
- `content_hash` ；
- `last_seen_at` ；
- `status` ： `active/missing/replaced/conflict`。

MAP-ANC-001

对象身份必须来自 `object_id` ，不得来自页面标题、块路径或正文文本哈希。

MAP-ANC-002

一个对象最多拥有一个 active `primary_text` Anchor，但可拥有多个 `source/context/event` Anchor。

3. 正文锚点

主正文可以是：

- 一个块及其子树；
- 一个页面中的指定区域；
- 一组由插件管理的块。

首版 **SHOULD** 以一个稳定 Block UUID 作为主正文锚点，避免整页被对象独占。

4. 物理移动

块移动后 UUID 保持时，Anchor 不变，只更新页面和路径缓存。若移动导致 UUID 变化，系统必须通过用户确认或内容/事件线索建立 **replaced_by**，不得静默猜测。

5. 正文替换

用户接受内容改写时：

1. 校验 Anchor 和正文版本；
2. 记录原始内容到 Audit Store；
3. 在原位置替换正文；
4. 保留 Block UUID；
5. 更新 content hash；
6. 记录 **content_rewritten** 事件。

MAP-RWT-001

仅文字替换不得改变对象类型、主归属和物理位置。

6. 拆分与合并块

当一个主正文块拆成多个块：

- 原 Anchor 可以继续指向容器块；
- 子块作为 context Anchor；
- 若主正文身份转移，必须生成 Anchor 迁移操作。

当多个块合并：

- 保留 canonical primary Anchor；
- 其他 Anchor 标记 replaced；

- 来源和历史关系不删除。

7. Journal 策略

Capture 默认可以留在 Journal。正式化有三种合法结果：

1. **原地正式化**：替换原文，建立对象锚点；
2. **原地正文 + 语义关联**：正文仍在 Journal，对象主归属在领域模型；
3. **移动正文**：经独立确认后移动到项目位置，同时保留 Journal source Anchor。

默认优先 1 或 2，避免为了整理而打断时间流。

8. 页面与对象关系

页面标题可以绑定 Project/Area 的导航入口，但页面不等于对象。一个项目页面可以包含：

- 项目主正文锚点；
- 生成视图块；
- 事件引用；
- 文件入口；
- 普通非对象笔记。

MAP-PAGE-001

重命名页面不得改变 Project 身份。

MAP-PAGE-002

删除项目页面不等于删除 Project；应产生 missing anchor 冲突并提供重建入口。

9. Markdown 中的隐藏数据

若 Logseq/插件能力允许，最小映射元数据可以存放于插件管理文件或隐藏属性。禁止把完整领域对象序列化到每个正文块。

推荐目录概念：

```
logseq/  
task-system/  
  schema.json  
  anchors.jsonl  
exports/
```

具体路径由插件实现和同步策略决定。

10. 失联处理

Anchor 失联时：

- 不自动删除对象；
- 标记 `missing` 和 `CONFLICT`；
- 尝试通过 UUID 索引、事件记录和用户选择恢复；
- 提供“重新绑定”“保留无正文对象”“归档对象”选项；
- 任何模糊自动匹配只能作为建议。

版本、审计与回滚规范

本章回答

保存正文替换、状态变化和语义提交的完整历史。

核心结论

当前正文保持唯一，修改仍可追溯、撤销与恢复。

1. 信任目标

用户必须能够回答：Agent 改了什么、为什么改、我批准了什么、最终写到了哪里、能否撤销。审计系统服务信任和恢复，不应成为正文噪声。

2. DomainEvent

每个正式变化必须产生不可变事件，至少包含：

- event_id;
- timestamp;
- actor: user/agent/system/import;
- object_id;
- operation_type;
- semantic_commit_id;
- before_version / after_version;
- payload 或 patch;
- source_proposal_id;
- rule_refs;
- reversible 标志。

AUD-EVT-001

事件写入必须与领域状态提交原子一致：状态成功而事件失败、或事件成功而状态失败均不合法。

3. 正文版本

正文替换必须保存：

- 原始文本；
- Agent 提议文本；

- 用户编辑后的最终文本；
- 接受/拒绝的操作；
- 原文哈希和最终哈希；
- Anchor；
- 时间和操作者。

可使用完整快照加差异，首版优先可靠性而非存储节省。

AUD-TXT-001

正式内容替换后，原始文本不必在正常正文并列显示，但 **MUST** 能从审查记录或历史视图恢复。

4. SemanticCommit 记录

每次提交记录：

- 被接受的操作；
- 被编辑的操作；
- 被拒绝/暂缓的操作；
- 操作依赖图；
- 每个权威存储的写入结果；
- 失败补偿结果；
- 用户可读摘要。

5. 回滚层级

5.1 单操作撤销

适用于独立、无下游依赖的操作，例如恢复一次正文改写。

5.2 整次提交撤销

按逆序执行补偿操作，恢复 Domain Store、Logseq 正文和关系。

5.3 对象时间点恢复

从事件重建某个版本，并生成新的恢复提交；不得删除中间历史。

AUD-RBK-001

回滚本身也是新事件，不得改写或删除旧事件。

6. 不可逆外部副作用

发送邮件、推送外部系统、删除外部文件等不可逆操作不属于首版默认能力。未来若支持，必须：

- 明确标记不可逆；
- 单独确认；
- 不与普通语义提交捆绑；
- 记录外部响应证据。

7. 保留策略

- 领域事件：长期保留；
- 正文旧版本：长期保留或基于明确策略压缩；
- 未提交 Proposal：可在用户设定期限后归档；
- 生成缓存：可随时清理；
- 敏感内容：允许用户执行显式永久删除，但必须清楚告知不可恢复。

8. Git 的角色

Git **MAY** 作为 Logseq 文件层额外版本保障，但不得替代领域事件：

- Git 不理解语义操作；
- 数据库状态可能不在同一提交；
- 用户未必每次及时提交。

系统 **SHOULD** 在关键 SemanticCommit 后生成一致性快照或可关联的 Git commit metadata。

9. 审计视图

正常工作视图不显示完整日志。对象抽屉可显示简洁活动流；独立历史页面提供：

- 按时间、对象、操作和 Agent 筛选；
- 修改前后 Diff；
- SemanticCommit 分组；
- 回滚入口；
- 规则依据。

捕获、分诊与正式化流程规范

本章回答

定义随手输入如何经过分诊与审查成为正式对象。

核心结论

记录时低摩擦，重要信息最终得到良好安排。

1. 目标

捕获流程必须允许低摩擦输入，同时保证重要内容最终能够进入正式正文、对象和工作流。系统不追求“所有内容立即整理”，而追求“重要内容最终得到可靠处理”。

2. 捕获入口

合法入口包括：

- Logseq Journal 中带轻量标记的块；
- 当前块/选中块的“送入 Inbox”；
- 插件快速输入；
- Agent 对话中的明确捕获指令；
- 外部文本、文件或剪贴板导入；
- 对现有对象的进展更新。

CAP-IN-001

捕获时 **MUST NOT** 强制用户填写对象类型、归属、优先级、截止时间和完整模板。

CAP-IN-002

系统必须保存原始文本、来源位置、时间和捕获方式。

3. 轻量提示

未处理 Capture 可显示一个低干扰标记，例如细边框、图标或弱色点。不得在每个 Capture 下自动插入大型模板或多行属性。

用户可选择：

- 暂时忽略；

- 立即调用 Agent；
- 稍后批量审查；
- 标为普通笔记无需处理。

4. 分诊分类

Agent 对 Capture 首先判断其处理路径，而不是直接创建 Task：

1. 新工作对象；
2. 现有对象的新进展；
3. Decision 候选；
4. Output/Resource/ExternalArtifact；
5. 纯上下文，无需行动；
6. 重复内容；
7. 信息不足，继续 Capture；
8. 无需保留或用户明确拒绝。

CAP-TRI-001

分诊结果必须包含依据和不确定项。

CAP-TRI-002

同一 Capture 可以同时更新现有对象并产生新 Task，但每项操作独立审查。

5. 正式化阶段

5.1 理解

提取明确事实、约束、时间、相关人、动作、结果、推测和未知。

5.2 匹配

搜索可能的现有对象、重复输入、相关项目和来源。

5.3 生成 Proposal

Proposal 包含正文改写和语义操作，不直接写入正式层。

5.4 用户审查

用户可整体查看并分项接受、编辑、拒绝、暂缓。

5.5 SemanticCommit

系统校验版本、依赖和规则，原子写入各权威层。

5.6 反馈

清晰报告：做了什么、没做什么、内容现在在哪里、是否仍有待确认项。

6. 原地替换策略

默认情况下，用户接受正式文字后，系统在原 Capture 位置替换正文，保留 Anchor 和历史。只有在用户独立接受 `move_content` 时才移动。

CAP-FRM-001

正式内容替换原始正文并不自动等于 Capture 成为新对象；它也可能只是现有对象的进展或普通正式笔记。

7. 批量审查

审查中心支持按以下条件筛选：

- 捕获时间；
- 所在页面/Journal；
- 建议归属；
- Proposal 类型；
- 是否影响正式对象；
- Agent 置信度；
- 暂缓原因。

批量接受只允许低风险、同质操作。主归属、类型迁移、删除、项目完成等不得无差别批量接受。

8. 处理时机

系统支持三种节奏：

- **即时**：用户主动对当前块调用；
- **集中**：用户在 Inbox 审查；
- **健康检查**：Agent 发现高价值问题后提示。

默认不得在每次输入后自动弹出审查窗口。

9. 完成定义

Capture 仅在以下情况进入 RESOLVED：

- 已转换为正式内容/对象并成功提交；
- 已合并进现有对象；
- 用户确认它只是普通正式笔记；
- 用户明确标记无需处理。

生成 Proposal 但尚未提交不算解决。

Agent 语义建议与提交协议

本章回答

把 Agent 的一次处理拆成可组合、可依赖、可原子提交的语义操作。

核心结论

AI 改写不是一个按钮，而是一组可以分别决定的正式变化。

一次 Agent 建议 = 一组可独立决定、具有依赖关系的语义操作

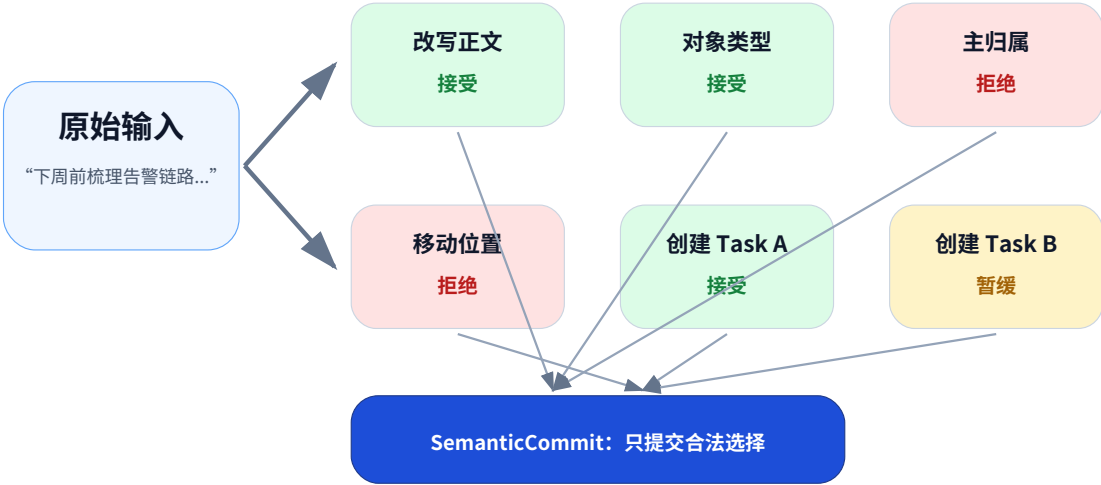


图 8 Proposal 是语义操作图；SemanticCommit 只应用用户选择后仍合法的操作。

1. 核心设计

Agent 不直接“修改一段笔记”，而是生成一个 Proposal。Proposal 由一组类型明确、可解释、可拆分的 SemanticOperation 构成。用户审查后，接受的操作组成 SemanticCommit，由确定性执行器原子提交。

2. Proposal 结构

Proposal 至少包含：

- proposal_id;
- source anchors / object ids;
- summary;
- extracted facts;
- assumptions;

- uncertainties;
- candidate operations;
- operation dependency graph;
- affected authorities;
- risk level;
- rule references;
- generated_at / model metadata。

COM-PROP-001

Proposal 是建议，不得被查询视图当作正式事实。

3. 标准操作类型

3.1 内容操作

- `rewrite_content`：改写正式文字；
- `append_content`：补充内容；
- `replace_summary`：更新当前摘要；
- `mark_superseded_text`：将旧结论降为历史。

3.2 对象操作

- `create_object`；
- `change_object_type`；
- `split_object`；
- `merge_objects`；
- `archive_object`；
- `soft_delete_object`。

3.3 关系操作

- `set_primary_owner`；
- `add_relation`；
- `remove_relation`；
- `rebind_anchor`；
- `move_content`。

3.4 workflow操作

- `change_phase`；

- `set_condition` ;
- `set_due_at` ;
- `set_review_at` ;
- `set_waiting_for` ;
- `create_next_action` ;
- `complete_object` 。

3.5 资产操作

- `register_output` ;
- `link_external_artifact` ;
- `create_file_workspace` 。

4. 操作契约

每个操作必须声明：

- `operation_id`;
- `type`;
- `human_summary`;
- `target`;
- `before`;
- `proposed_after`;
- `evidence`;
- `confidence`;
- `rule_refs`;
- `dependencies`;
- `conflict_group`;
- `risk`;
- `editable_fields`;
- `reversible`;
- `write_authority`。

COM-OP-001

文字改写、对象分类、主归属、物理位置、关系和状态必须是独立操作。

COM-OP-002

操作之间若存在语义依赖必须显式表达。例如创建 Task 可能依赖 create_object，但不必依赖 move_content。

5. 风险等级

Low

可再生成摘要、添加弱关联、提取候选日期但不提交、生成视图。

Medium

创建 Task、修改 Task Phase、设置 Waiting、改写正文、更新当前摘要。

High

改变主归属、对象类型迁移、合并、拆分、完成 Project、归档、删除、移动大量内容。

High 操作必须显式单独确认，不得预选批量接受。

6. 部分接受语义

用户可以：

- 接受 proposed_after；
- 编辑 proposed_after 后接受；
- 拒绝；
- 暂缓；
- 替换为其他操作，例如选择另一个主归属；
- 请求 Agent 重新生成该操作。

被拒绝操作的依赖者必须重新校验：

- 若依赖是强依赖，依赖者自动取消勾选并说明；
- 若依赖是弱依赖，依赖者可以独立提交；
- 系统不得暗中补齐被拒绝的操作。

7. SemanticCommit

提交步骤：

1. 冻结用户选择和编辑结果；
2. 校验所有对象/正文版本；

3. 构建最终操作 DAG；
4. 执行领域层预校验；
5. 创建恢复点；
6. 按权威存储执行；
7. 写入 DomainEvent；
8. 验证跨层一致性；
9. 返回用户可读结果；
10. 失败则补偿或保持未提交。

COM-ATM-001

一次 Commit 必须表现为“全部已确认操作成功”或“没有任何正式变化”。若跨系统无法实现真实事务，必须采用可恢复的 saga 并向用户暴露状态。

8. 版本冲突

Proposal 生成后正文或对象变化，提交必须停止并显示：

- 旧版本；
- 当前版本；
- 原建议；
- 可重新生成、手工合并或放弃。

禁止用 Agent 旧建议覆盖用户的新修改。

9. Agent 输出要求

Agent 必须区分：

- **facts**：来源明确的事实；
- **extractions**：从文本直接提取；
- **inferences**：有依据的推断；
- **recommendations**：推进建议；
- **unknowns**：无法判断；
- **operations**：计划执行的正式变化。

10. 示例

原输入：

下午领导说下周前最好把 Graylog 告警到外部推送这部分梳理清楚，之前脚本应该还能用，但最好再找个真实事件跑一下。

Proposal 可以包含：

1. `rewrite_content`：生成连贯正式文本；
2. `create_object(MiniProject)`：梳理告警链路；
3. `set_primary_owner`：建议归入 Graylog Project；
4. `create_object(Task)`：静态架构梳理；
5. `create_object(Task)`：真实事件验证；
6. `set_due_at`：候选截止日期；
7. `move_content`：建议移动到项目页。

用户可以接受 1、2、4，拒绝 3、5、7，并修改 6。提交后 MiniProject 处于待确认归属，正文原地替换，不创建真实事件 Task，也不移动块。

审查与部分接受规则

本章回答

规定用户怎样接受、编辑、拒绝或暂缓 Agent 建议。

核心结论

接受文字修改，不代表接受归属、位置和对对象创建。

1. 审查目的

审查不是让用户逐字校对 LLM，而是让用户控制正式语义变化。界面必须清楚展示“文字怎么变”和“系统准备做什么”两类差异。

2. 审查层次

2.1 文本层

显示修改前、Agent 建议、用户编辑后的最终文本。支持 inline diff 和完整预览。

2.2 语义层

显示对象类型、归属、状态、关系、任务创建和位置移动等操作。

2.3 影响层

说明受影响对象、视图、文件和下游操作。

REV-UI-001

界面不得只提供“全部应用”而隐藏语义操作。

3. 默认选择策略

- Low 风险操作 **MAY** 默认选中；
- Medium 操作可根据用户偏好选中，但必须清晰可见；
- High 操作 **MUST** 默认不选中并单独确认；
- 不确定性高的操作不得默认选中。

4. 部分接受

REV-PART-001

每个 SemanticOperation 必须可独立接受、编辑、拒绝或暂缓，除非其契约声明强依赖。

REV-PART-002

内容修改与位置调整必须分别审查。

REV-PART-003

内容修改与主归属必须分别审查。

REV-PART-004

创建父对象不意味着必须接受 Agent 生成的所有子任务。

REV-PART-005

用户编辑 Agent 建议后，最终内容视为用户确认版本，审计中同时保留 Agent 原建议。

5. 暂缓与拒绝

拒绝

表示当前建议不应执行。系统可记录拒绝原因用于 Skill 评估，但不得把拒绝转成另一项隐式操作。

暂缓

Proposal 保持待审查，允许设置复查时间。暂缓不应让原始 Capture 假装已正式化。

永久忽略

用户可将内容标为普通笔记或无需管理，Capture 进入 RESOLVED/DISMISSED。

6. 依赖处理

审查界面必须展示依赖。例如：

- `create_task` 依赖 `create_object` ；
- `move_content` 不依赖 `rewrite_content` ；
- `change_phase to READY` 依赖完成标准已存在。

拒绝前置操作时，界面应说明哪些操作将失效，并允许用户修改使其重新合法。

7. 批量审查

允许批量：

- 接受相同类型的低风险文本格式调整；
- 将多个明确 Capture 标为普通笔记；
- 批量增加弱关联；
- 批量处理过期的 Proposal。

禁止无差别批量：

- 改变主归属；
- 对象类型迁移；
- Project 完成或归档；
- 合并/删除；
- 大规模移动正文；
- 处理存在版本冲突的建议。

8. 审查反馈

提交后必须显示：

- 已执行；
- 用户修改后执行；
- 未执行；
- 因依赖或冲突跳过；
- 最终正文位置；
- 新建/更新对象；
- 可撤销入口。

9. 降低审查疲劳

REV-FAT-001

系统不得为没有实质变化的内容生成 Proposal。

REV-FAT-002

纯语言改写若不能显著改善准确性、可读性或后续管理，不应打扰用户。

REV-FAT-003

系统 **SHOULD** 学习用户反复拒绝的建议类型，但学习结果只影响推荐和默认选择，不得绕过正式规则。

10. 信任保护

Agent 的解释不得使用“我已经帮你完成”描述尚未提交的建议。审查前应使用“建议”“计划”“将会”。提交成功后才可使用“已更新”。

注意力、呈现与 Agent

完整数据留在系统里，当前视图只呈现此刻真正需要的信息。

人通过有限的注意力窗口工作，而不是通过浏览完整数据库工作。

注意力场景与视图投影规范

本章回答

围绕捕获、执行、重入、等待、审查和成果整理设计视图。

核心结论

同一份数据根据注意力场景投影，而不是复制成多份状态。

同一事务对象，在不同注意力场景下产生不同投影

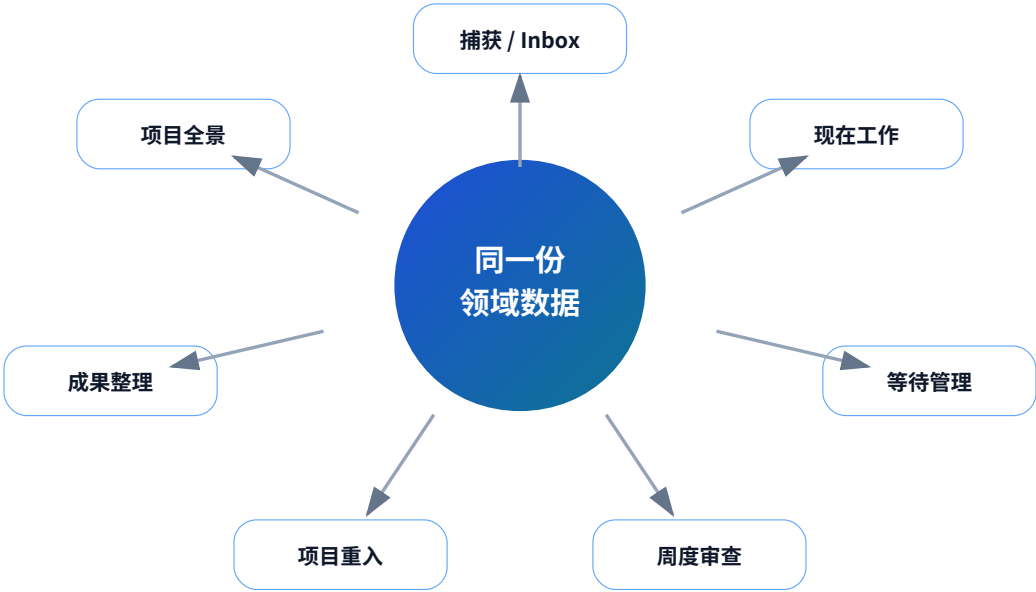


图 9 视图是同一领域数据在特定注意力场景下的投影。

1. 核心原则

用户不通过浏览完整数据库工作，而通过有限注意力窗口工作。视图是领域数据的投影，不是任务副本。

VIEW-BASE-001

每个视图必须声明用户目标、数据范围、排序、显示字段、隐藏字段和高频操作。

VIEW-BASE-002

同一对象在多个视图中必须引用同一 object_id，所有修改回到同一权威状态。

2. 捕获视图

用户目标

快速记录而不被分类打断。

默认显示

- 输入框或当前块；
- 极少量入口：保存、调用 Agent、稍后处理；
- 当前页面和时间作为隐含来源。

默认隐藏

类型、状态、所有可能项目、完整模板。

3. Inbox / 分诊视图

用户目标

将未处理输入转为正式内容、对象、进展或普通笔记。

显示

- 原始内容；
- 来源和时间；
- Agent 简短判断；
- 待审查操作数量和风险；
- 可能匹配的现有对象；
- 暂缓/忽略状态。

操作

打开 Proposal、接受低风险建议、标记普通笔记、合并重复、暂缓。

排序优先级：即将过期的明确需求 > 可能影响活跃对象的进展 > 长期未处理 > 普通想法。

4. “现在工作” 视图

用户目标

在不重新规划整个系统的情况下开始行动。

显示

- 已承诺 Active 且 Actionable 的工作；
- Ready 中当前最值得开始的少量对象；
- 到期/复查信号；
- 必要的完成标准和最小上下文；
- 快速更新、等待、阻塞、完成入口。

禁止

- 默认显示全部 Project 历史；
- 用几十项长列表制造选择压力；
- 把优先级作为唯一排序依据。

排序 **SHOULD** 综合：承诺状态、截止、依赖影响、复查、停滞、上下文可用性和用户当前 Focus。

5. 单个 Task 工作视图

显示：

- 现在要完成什么；
- 完成标准；
- 为什么需要做；
- 当前下一动作；
- 阻塞/等待；
- 必要来源和文件；
- 最近相关事件。

操作：开始、记录进展、标等待、标阻塞、完成、请求 Agent 建议、进入上层上下文。

6. Project 重入视图

用户目标

从中断状态恢复行动能力，而不是阅读全文摘要。

必须显示

1. 项目目的和目标结果；
2. 边界；
3. 当前阶段和执行条件；
4. 最近一段时间的关键变化；

- 5. 已确认结论；
- 6. 当前推进、待决策和阻塞；
- 7. 哪些旧信息已失效；
- 8. 建议的第一个恢复动作；
- 9. 最值得打开的少量上下文入口。

VIEW-RE-001

重入包必须以恢复行动为目标，不得只是把所有事件压缩成一篇长摘要。

7. Project 工作台

默认显示：

- 当前状态叙事；
- Active/Ready 下层对象；
- 当前阻塞和等待；
- 待决策；
- 关键成果；
- 最近重要事件；
- 全部内容入口。

已完成任务、历史事件、完整文件清单和内部属性默认折叠。

8. 等待与阻塞视图

Waiting

必须显示：在等谁/什么、期待结果、开始等待时间、复查时间、关联项目、最近跟进。

Blocked

必须显示：阻碍、是否有人处理、可采取的解除动作、影响对象、持续时间。

VIEW-WAIT-001

Waiting 和 Blocked 必须分开呈现，因为前者主要需要复查，后者主要需要解除阻碍或重新决策。

9. 周期审查视图

按以下问题组织，而不是按项目机械罗列：

- 哪些 Active 对象没有下一动作；
- 哪些等待已经到复查时间；
- 哪些对象可能停滞；
- 哪些范围正在膨胀；
- 哪些项目已经事实上完成；
- 哪些成果尚未登记；
- 哪些 Capture 长期未处理；
- 哪些归属或映射存在冲突。

操作应包括：确认健康、调整下一步、暂停、迁移、归档、请求 Agent 分析。

10. 成果整理视图

显示最近完成工作、候选产物、可复用经验、尚未登记的文件、可转为周报/文档/模板的内容。

11. 自定义视图

用户可按对象类型、Area、Project、Phase、Condition、Signal、日期、人员、标签、全文和关系构建查询。

VIEW-CUS-001

自定义视图只保存查询和投影，不得改变对象归属。

VIEW-CUS-002

Logseq Query 可以作为一种适配实现，但领域视图定义不得绑定某一种查询语法。

12. 视图密度

每种视图支持：

- Compact：标题、状态、下一步；
- Normal：增加简短背景和信号；
- Expanded：完整工作接口。

用户可保存偏好，但系统在窄屏、侧栏和全页中可选择不同默认密度。

自然语言呈现与信息密度规范

本章回答

建立自然语言、结构密度、强调与摘要更新规范。

核心结论

自然语言是主要阅读界面，结构只在真正提供价值时出现。

1. 目标

正式内容应当像经过良好整理的真实工作笔记，而不是数据库导出、机械模板或空洞报告。结构服务信息，不能成为信息本身。

2. 基本风格

TXT-STYLE-001

主体内容 **SHOULD** 使用连贯自然语言解释因果、进展和下一步；只有当项目适合比较、枚举或执行时才使用列表。

TXT-STYLE-002

不得把完整信息机械拆成“时间、地点、人物、目的、过程、结果”属性清单，除非用户正在查看属性面板或该格式具有明确操作价值。

TXT-STYLE-003

语言应具体、克制、可验证。避免“全面赋能”“进一步提升”等没有事实含量的表述。

TXT-STYLE-004

正文应区分：已确认事实、当前判断、推测、建议和待确认问题。

3. 结构生成原则

TXT-STRUCT-001

只有存在有效内容且结构能改善理解或操作时，栏目才可出现。

TXT-STRUCT-002

禁止生成空的“背景”“风险”“下一步”“相关资料”等标题。

TXT-STRUCT-003

简单 Task 可以只有一段文字和隐藏属性；不得为了符合 Project 模板而膨胀。

TXT-STRUCT-004

结构深度默认不超过三级。更深细节应通过折叠、关联页或文件工作区承载。

4. 信息密度层级

D1: 标题/一句话

适用于列表和 Compact View。必须能够识别对象和主要动作。

D2: 工作摘要

1-3 个短段或少量列表，表达目标、当前状态、下一步和阻塞。适用于抽屉和项目首页。

D3: 完整上下文

保留背景、过程、决策、证据和资源入口，适用于详细页和重入。

D4: 历史证据

完整事件、原始版本、日志和调试信息，只在需要追溯时打开。

TXT-DEN-001

系统不得在 D1/D2 视图中直接倾倒 D3/D4 内容。

5. 强调规则

可使用：

- **加粗**：当前结论、关键约束、明确动作；
- 颜色/徽标：Phase、Condition、Signal；
- 引用块：原始要求或外部证据；
- 弱化文本：历史背景和低优先细节。

不得用大量颜色、Emoji 和粗体制造视觉竞争。一个短段中通常只应有少量真正重点。

6. 不同对象的正文要求

Task

自然表达“做什么、为什么、完成到什么程度”。背景只保留避免误解所需内容。

MiniProject

先说明有限目标和当前状态，再展开局部推进结构。不要默认生成项目级长篇章。

Project

项目首页必须克制，详细历史和资源通过入口展开。当前状态段应回答“最近发生了什么以及现在意味着什么”。

Area

强调责任标准和当前活跃项目，不把长期 Area 写成无穷尽任务列表。

7. 时间与人名

时间、相关人、截止和复查等属性默认在界面组件展示。若它们影响叙事，应自然写入正文，例如：

需要在下周前完成整体链路说明，目前仍等待厂商确认接口范围。

而不是：

截止时间：下周；等待对象：厂商；任务：完成说明。

8. Agent 改写边界

Agent 可以：

- 修复语音识别和明显语病；
- 补全原文中已经隐含但未说清楚的逻辑连接；
- 合并重复；
- 突出明确重点；
- 调整段落顺序。

Agent 不得：

- 添加没有来源的事实；
- 把建议写成既定安排；
- 过度删减背景；
- 因追求简洁丢失重要不确定性；
- 因追求完整添加空洞结构。

9. 摘要更新

当前摘要应采用“增量理解”，而不是每次重新生成一篇风格不同的文章。更新时优先：

1. 保留仍有效的核心结论；
2. 用新事件替换过时状态；
3. 删除已不影响当前行动的细节；
4. 将历史证据留在事件层；
5. 保持语言和结构稳定。

10. 质量检查

正式内容提交前应检查：

- 是否忠实原意；
- 是否存在杜撰；
- 是否丢失关键背景；
- 是否结构大于内容；
- 是否能快速识别当前重点；
- 是否区分事实和推断；
- 是否适合未来重入；
- 是否符合所在对象粒度。

结构显隐与渐进披露规范

本章回答

通过行内标记、悬浮卡、抽屉和独立页面渐进披露信息。

核心结论

项目页瘦身不是删除信息，而是改变默认投影。

1. 目的

系统不是简单“隐藏属性”，而是让同一对象按注意力场景投影不同信息。用户应从最少必要内容逐步进入完整上下文。

2. 四级界面

2.1 行内标记

适合：Capture 状态、Phase、Waiting、Overdue 等轻量提示。

要求：不改变正文阅读节奏；禁止显示大量字段。

2.2 悬浮卡

适合快速查看：

- 对象类型；
- Phase/Condition；
- 主归属；
- 下一步；
- 截止/复查；
- 相关人；
- 打开抽屉入口。

悬浮卡只查看，不承担复杂编辑。

2.3 侧边抽屉

适合单对象工作：

- 编辑标题和状态；
- 查看/修改关系；
- 更新进展；
- 审查当前块 Proposal；

- 浏览必要上下文和文件；
- 查看简洁活动流。

抽屉应保持原页面可见。

2.4 独立页面

适合需要集中注意力的任务：

- Inbox 和批量审查；
- 现在工作；
- Project 重入；
- 周期审查；
- 项目全景；
- 自定义视图；
- 审计与回滚。

DISC-001

查看越轻，界面越靠近原文；决策越复杂，界面越独立。

3. 默认隐藏内容

正常正文默认隐藏：

- 内部 ID；
- 完整关系清单；
- 审计元数据；
- Agent 模型信息；
- 已完成的全部下层对象；
- 空属性；
- 低价值内部标签；
- 完整文件 manifest。

4. 按内容出现

DISC-002

无有效风险时不显示风险栏目；无 Waiting 时不显示等待区域；无 Decision 时不显示决策标题。

DISC-003

结构的存在由数据判断，显示由视图判断，两者不得混为一体。

5. 用户定制

用户可以保存：

- 各视图密度；
- 默认展开区域；
- 属性显示偏好；
- 自定义 Query；
- Focus 模式规则。

但系统应提供合理默认，不能要求用户先配置复杂仪表盘才能工作。

6. Focus 模式

进入某个 Task/MiniProject 的 Focus 模式时，只显示：

- 当前目标；
- 当前动作；
- 完成标准；
- 必要上下文；
- 相关文件；
- 更新进展和退出口。

所有其他项目导航、历史和非必要字段暂时隐藏。

7. 可访问性和可理解性

隐藏不得造成信息不可发现：

- 必须有清晰展开入口；
- 图标必须有文字说明；
- 颜色不能是唯一状态表达；
- 键盘可完成核心操作；
- 抽屉和弹窗应可返回原上下文。

8. Logseq 页面瘦身策略

项目页建议只保留：

- 1. 项目定位；
- 2. 当前状态；
- 3. 当前推进；
- 4. 阻塞/等待；
- 5. 关键成果；
- 6. 详细内容入口。

这些区域也是动态投影：没有内容则不渲染，完整历史保留在事件和详细视图。

Agent 角色、权限与自动化边界

本章回答

定义 Agent 的角色、权限等级、自动化范围和事实保真要求。

核心结论

Agent 负责理解和建议，确定性代码负责守住边界。

1. 角色模型

Agent 可以承担多个角色，但每次 Proposal 必须声明当前角色和作用范围。

1.1 捕获解释者

识别事实、动作、约束、不确定性和可能的现有对象。

1.2 内容编辑者

改善正式语言、可读性和信息密度，不改变未获批准的语义事实。

1.3 归属建议者

根据对象边界和历史建议主归属及关联。

1.4 项目秘书

维护当前摘要、最近变化、等待、阻塞、成果和重入包。

1.5 workflow 分析者

发现缺少下一步、非法状态、停滞和到期复查。

1.6 层级治理者

建议升级、降级、拆分、合并和归属迁移，并给出规则证据。

1.7 成果整理者

识别可登记 Output、经验和可复用资产。

2. 权限等级

Level 0：只读分析

搜索、汇总、计算信号、生成临时视图。不得写正式层。

Level 1：创建 Proposal

可生成任何受支持建议，但不执行。

Level 2：自动执行低风险操作

仅可执行用户预先授权且可逆的操作，例如刷新视图缓存、计算 Signal、保存草稿 Proposal。

Level 3：经确认执行正式提交

用户审查后调用确定性提交器。

AGT-AUTH-001

Agent 不直接获得数据库任意写权限；必须通过领域命令/API。

3. 强制人工确认操作

- 改写正式正文；
- 新建正式 Project/MiniProject；
- 改变主归属；
- 对象类型迁移；
- 移动正文；
- 完成/取消/归档 Project 或 Area；
- 合并、拆分、删除；
- 覆盖存在冲突的内容；
- 任何不可逆外部副作用。

用户可为特定低风险场景建立授权规则，但不能取消系统的高风险边界。

4. 自动化允许范围

Agent/规则可以自动：

- 计算 Overdue、ReviewDue、Stale 等 Signal；
- 生成可丢弃摘要草稿；

- 建立全文索引；
- 提示 Anchor 失联；
- 在用户明确输入“完成了任务 X”后生成完成 Proposal；
- 归并明显相同的候选搜索结果，但不得正式合并对象。

5. 不确定性表达

Agent 输出必须把判断分为：

- Confirmed：用户或权威来源明确；
- Extracted：从原文直接提取；
- Inferred：基于上下文推断；
- Suggested：为了推进提出；
- Unknown：信息不足。

AGT-TRUTH-001

Inferred/Suggested 不得在正文中使用确定语气，除非用户确认后成为正式判断。

6. 杜撰防护

Agent 必须：

- 引用输入、对象或事件作为依据；
- 不补充具体日期、人名、技术结论和完成情况；
- 在信息不足时减少结构而不是填空；
- 发现冲突时呈现冲突，不选择更“好看”的版本。

7. 主动提醒边界

允许主动提醒：

- 高优先截止临近；
- Waiting 到复查时间；
- Active Project 长期无下一步；
- 数据冲突或 Anchor 失联；
- 高置信度重复对象；
- 项目可能已完成但未收尾。

不应主动打扰：

- 每条普通笔记的润色；
- 低价值标签建议；
- 可以等到周期审查处理的格式问题。

8. Agent 更换和可复现性

Proposal 必须记录模型、Skill 版本、输入范围和规则版本。更换模型后，正式领域约束和已确认内容不受影响。

9. 安全失败

Agent 不可用时，用户仍能：

- 捕获；
- 手动创建和更新对象；
- 查看工作台；
- 执行合法状态变化；
- 导出和恢复数据。

Agent 是增强层，不得成为系统基本可用性的单点故障。

Skill 架构与编写规范

本章回答

把事务语义、内容编辑、归属、 workflow、迁移和视图拆成可测试 Skill。

核心结论

Skill 是领域操作手册，不是一份无限膨胀的总提示词。

1. 目标

Skill 是 Agent 对领域规范的可执行解释层，不是唯一规则来源。确定性约束仍由代码执行。Skill 应模块化、可测试、版本化，并能引用稳定规则 ID。

2. Skill 分层

2.1 Foundation Skill

加载项目原则、术语和真值分级，所有其他 Skill 必须依赖。

2.2 Semantic Classification Skill

判断 Capture、Task、MiniProject、Project、Area、Decision、Output 和 Resource，输出证据、反证和不确定性。

2.3 Content Editing Skill

负责忠实改写、信息密度和结构克制。

2.4 Ownership Skill

判断主归属、关联、来源和上下文，不处理物理移动。

2.5 Workflow Skill

建议 Phase、Condition、下一步、Waiting、Blocked 和复查日期。

2.6 Migration Skill

建议升级、降级、拆分、合并，并引用迁移规则。

2.7 View Skill

生成项目重入、当前工作、周审查和成果整理投影。

2.8 Review Skill

将建议拆为 SemanticOperation，建立依赖和风险等级。

2.9 External Context Skill

处理文件工作区、Git、Zotero 和外部 Artifact 关系。

3. Skill 契约

每个 Skill 必须声明：

- skill_id 和版本；
- 目的；
- 输入 schema；
- 输出 schema；
- 引用规则；
- 可调用领域查询；
- 禁止执行的操作；
- 不确定性策略；
- 黄金样例；
- 失败模式。

SKL-001

Skill 输出必须结构化，不能只返回一段无法审计的自然语言。

SKL-002

Skill 不得复制并修改上游规则；应引用规则 ID。必要的摘要必须声明仅为解释。

4. 输出规范

推荐统一输出：

```
analysis:
  facts: []
  extractions: []
  inferences: []
  unknowns: []
operations:
  - operation_id:
    type:
```

```
target:
proposed_after:
evidence: []
rule_refs: []
confidence:
risk:
dependencies: []
user_summary:
```

5. 上下文获取

Skill **SHOULD** 按需读取：

1. 当前 Capture/对象；
2. 直接主归属接口；
3. 最近相关事件；
4. 候选重复或归属对象；
5. 必要的完整正文。

禁止默认读取整个 Graph 后再“凭感觉”判断，以降低噪声和成本。

6. 语言风格

Skill 必须遵守 **14-自然语言呈现与信息密度规范.md**，尤其是：

- 不生成空标题；
- 不用属性清单替代叙事；
- 不把推断写成事实；
- 不因模板要求增加无价值内容；
- 保留用户的专业表达和关键细节。

7. 测试

每个 Skill 至少需要：

- 正常案例；
- 信息不足案例；
- 边界类型案例；
- 杜撰诱导案例；
- 用户部分拒绝案例；
- 上下文冲突案例；
- 多次运行稳定性案例。

评估维度：事实保真、分类准确、规则引用、结构克制、建议可执行、操作可拆分和语言质量。

8. 版本治理

Skill 的重大语义变化必须：

- 增加版本；
- 更新黄金样例预期；
- 运行回归测试；
- 记录 ADR；
- 说明与领域规范版本兼容性。

9. 错误处理

信息不足时 Skill 应返回 `needs_clarification` 或低置信 Proposal，不得为了完成输出而填充猜测。若规则冲突，应返回冲突规则和最小化安全建议。

架构、插件与外部资源

Logseq 是首要入口，但不是领域语义本身。

领域模型保持独立，前端和适配器才能演进而不推翻系统骨架。

总体系统架构与领域数据模型

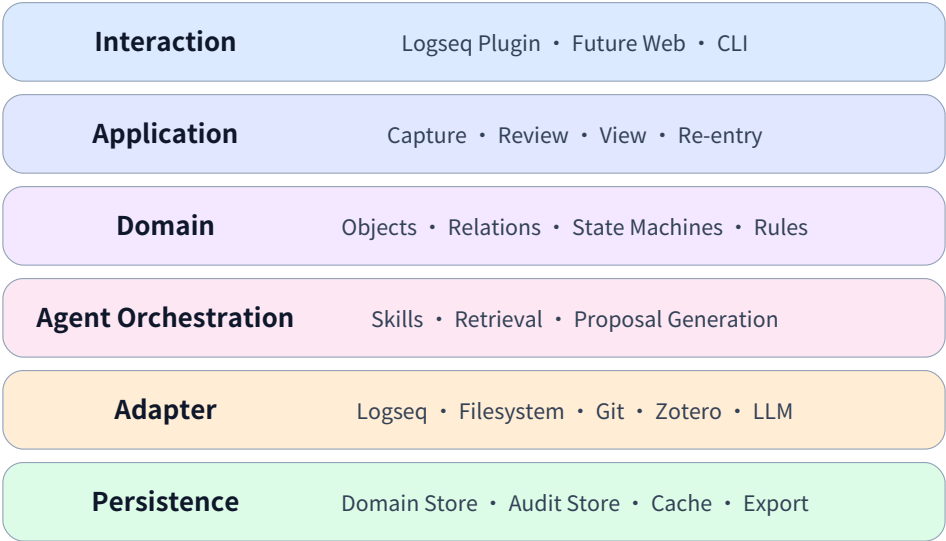
本章回答

给出前端、应用、领域、Agent、适配器和持久化的分层架构。

核心结论

MVP 可以内嵌运行，核心语义仍为未来服务化预留边界。

分层架构：把语义放在领域层，把 Logseq 放在适配与交互层



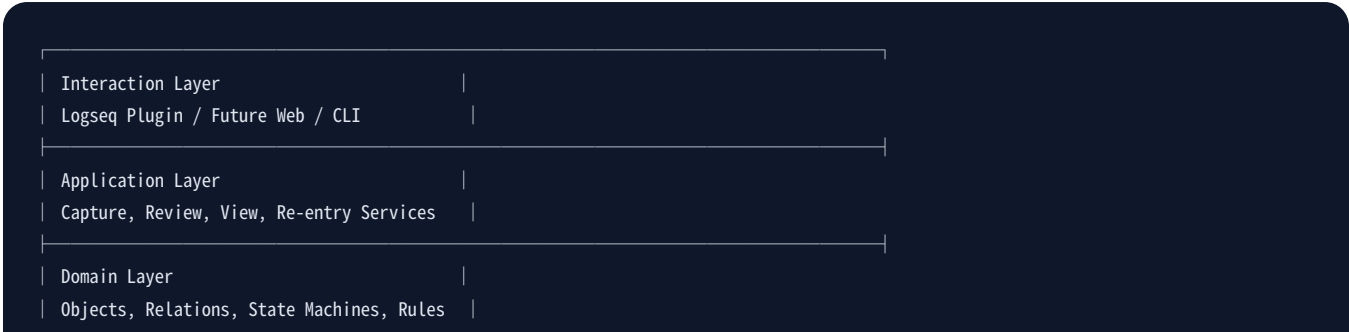
Domain Layer 不依赖 Logseq API; Agent 只能调用领域命令，不能直接写数据库。

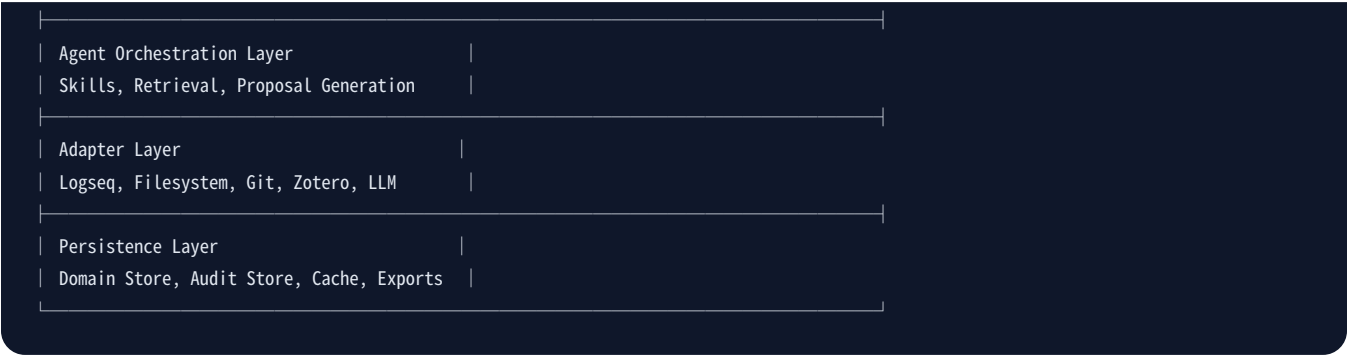
图 10 领域层保持前端无关，Logseq 通过 Adapter 与 Interaction Layer 接入。

1. 架构目标

系统必须本地优先、可追溯、可导出、前端可替换。Logseq 是首要交互入口，但领域模型、提交协议和审计不能依赖 Logseq DOM、页面路径或特定查询语法。

2. 分层架构





ARC-LAYER-001

Interaction 和 Adapter 层不得实现独立的对象语义或状态机。

ARC-LAYER-002

Domain Layer 不得依赖 Logseq API；只能依赖抽象 Anchor 和正文读写接口。

3. 推荐部署形态

3.1 MVP

- Logseq 插件 UI；
- 插件内 Application/Domain 模块；
- 本地 SQLite Domain/Audit Store；
- Logseq Adapter；
- 可选 LLM Provider；
- JSONL 导出。

3.2 演进形态

- 独立本地服务承载 Domain、Agent 和 API/MCP；
- Logseq 插件退化为一个前端适配器；
- CLI/Web 共享同一领域 API。

首版代码必须保持模块边界，使迁移不需要重写领域规则。

4. 核心实体

4.1 managed_object

```
object_id      TEXT PK
object_type    ENUM
phase         ENUM
condition      ENUM
version        INTEGER
primary_text_anchor FK nullable
created_at
updated_at
ended_at nullable
metadata_json  limited extensibility
```

对象专属数据可通过 `typed tables` 或 `validated JSON schema` 保存。禁止把所有语义都放入无法约束的 `metadata`。

4.2 object_relation

```
relation_id
from_object_id
to_object_id
relation_type
status
created_at
ended_at
metadata
```

唯一约束和循环检测由领域服务保证。

4.3 anchor

按 [08-Logseq映射与对象锚点规范.md](#)。

4.4 proposal / semantic_operation

保存待审查建议及依赖图。Proposal 与正式对象分表，查询默认排除。

4.5 semantic_commit

保存最终选择、执行状态和补偿信息。

4.6 domain_event

Append-only 审计事件。

4.7 view_definition

保存查询 DSL、投影、排序、密度和用户偏好。

4.8 external_artifact

保存 URI、类型、权威系统、版本/哈希、可用状态和元数据。

5. 对象专属字段建议

Task

- title;
- completion_criteria;
- next_action_text 或 child task;
- due_at;
- review_at;
- waiting_for_ref;
- priority_fact（可选，不等于注意力分数）。

MiniProject / Project

- purpose;
- target_outcome;
- scope_in;
- scope_out;
- completion_criteria;
- current_summary_anchor/cache;
- review_cadence;
- workspace_ref。

Area

- responsibility_statement;
- maintenance_standards;
- review_cadence。

6. Command 与 Query 分离

Domain API 使用明确命令：

- CaptureInput;
- CreateObject;
- ChangePhase;
- SetCondition;

- SetPrimaryOwner;
- AddRelation;
- ApplySemanticCommit;
- RebindAnchor;
- RegisterOutput;
- RollbackCommit。

查询 API:

- GetObjectInterface;
- QueryView;
- GetReentryPackage;
- ListAttentionSignals;
- GetAuditTrail;
- FindOwnershipCandidates。

ARC-CQRS-001

Agent 不得使用底层 SQL 直接修改状态，必须调用命令。

7. 规则引擎

规则分为：

- Schema 约束；
- 状态转换约束；
- 关系约束；
- 提交依赖约束；
- 动态 Signal 规则；
- 权限规则。

所有拒绝必须返回稳定 error code、相关 rule_id 和用户可读解释。

8. View DSL

视图 DSL 应支持：

- object_type;
- phase/condition/signal;
- primary ownership subtree;

- relation filters;
- full-text;
- date range;
- sort/group;
- projection fields;
- density;
- limit。

DSL 必须独立于 Logseq Query，可由 Adapter 转换或直接通过 Domain Store 查询。

9. 缓存和摘要

当前摘要、重入包和全文索引可以缓存，但必须记录来源版本。当相关对象、事件或正文变化时失效。

ARC-CACHE-001

缓存失效不得导致事实丢失；最坏情况是重新生成或暂时不显示。

10. 扩展点

- LLM Provider;
- Note Adapter;
- Artifact Provider;
- Notification Provider;
- Sync Backend;
- Skill Registry;
- View Renderer。

扩展只能通过稳定接口，不得绕过领域命令。

11. 性能原则

- 默认只读取当前对象接口和最近事件；
- 大图谱使用增量索引；
- UI 操作不应等待完整 Agent 推理；
- Proposal 生成可取消；
- 搜索和视图先返回确定性数据，再补充 Agent 分析。

12. 安全原则

- 本地数据默认不上传；
- 发送给外部 LLM 的上下文可预览和最小化；
- 敏感字段支持遮蔽；
- API/MCP 使用最小权限和审计；
- 插件不执行任意 Shell，外部文件操作通过受限服务。

CHAPTER 19 • IMPLEMENTATION

Logseq 插件与适配层设计

本章回答

定义 Logseq 插件在捕获、工作台、抽屉、审查和适配中的职责。

核心结论

插件是事务控制平面，不是把更多字段重新塞回正文。

1. 插件定位

插件是嵌入 Logseq 的事务控制平面和适配前端，不是“增强 TODO 样式”的插件，也不是完整领域规则的唯一宿主。

2. 核心模块

2.1 Capture Integration

- 当前块送入 Inbox;
- 快捷输入;
- Slash Command;
- 选中多块捕获;
- Capture 轻量标记。

2.2 Object UI

- 行内状态徽标;
- 悬浮卡;
- 侧边抽屉;
- 对象链接和打开命令。

2.3 Workspaces

- 现在工作;
- Inbox;
- Project 重入;
- Waiting/Blocked;
- 周审查;
- 成果整理;
- 自定义视图。

2.4 Proposal Review

- 原文/建议 Diff;
- SemanticOperation 列表;
- 部分接受;
- 依赖提示;
- 提交和撤销。

2.5 Logseq Adapter

- Block/Page 读取;
- UUID 和页面事件监听;
- 正文替换;
- 块移动;
- Anchor 校验;
- 页面嵌入视图。

3. 交互入口

3.1 高频快捷键

- 捕获当前块;
- 调用 Agent 处理当前块;
- 打开对象抽屉;
- 记录进展;
- 标记 Waiting;
- 完成当前 Task;
- 打开现在工作。

3.2 右键菜单

仅放上下文相关操作，避免菜单过长。例：送入 Inbox、关联现有对象、作为进展、打开审查。

3.3 命令面板

适合低频复杂动作，如迁移对象、重建 Anchor、打开审计。

4. UI 行为

PLG-UI-001

插件不得在正文中永久插入完整属性面板。

PLG-UI-002

关闭插件后，正式正文仍应保持人类可读；任务状态可能无法完整显示，但不得只剩不可理解的 ID。

PLG-UI-003

所有写入前显示最终影响；提交后显示结果和撤销。

5. 页面嵌入

项目页可以嵌入插件渲染块，例如：

- 当前状态；
- 当前推进；
- 阻塞等待；
- 成果入口。

这些是动态视图，不应将查询结果写成静态副本。插件不可用时，可显示简短占位和打开导出文件的说明。

6. Capture 标记

标记应：

- 视觉弱；
- 不改变 Markdown 正文；
- 可隐藏；
- 点击打开处理入口；
- 已解决后自动消失。

7. Review 抽屉与独立页面

单块 Proposal 使用抽屉，批量使用独立页面。两者共享相同 SemanticOperation 组件和提交 API，禁止实现两套不同规则。

8. Logseq 事件处理

监听：

- block changed;
- block deleted;
- block moved;
- page renamed;
- graph opened/closed。

事件先更新 Adapter 索引，再决定：

- 无影响；
- Anchor 缓存更新；
- 产生冲突；
- 生成 Proposal。

不得在用户每次键入时触发昂贵 Agent 调用。

9. File Graph / DB Graph 兼容

Adapter 接口至少定义：

- getBlock;
- getPage;
- replaceBlockContent;
- moveBlock;
- subscribeChanges;
- resolveUuid;
- renderWidget;
- getGraphIdentity。

File Graph 和 DB Graph 分别实现，领域层不得感知差异。

10. 离线和降级

无 Agent 时：手动捕获、编辑、状态流转、查询和审计可用。

Domain Store 暂不可用时：插件进入只读模式，禁止把状态临时写回散乱 Markdown 属性作为替代。

11. MVP 范围

必须实现：

1. Capture 当前块；
2. 独立 Task/MiniProject 对象；
3. Block UUID Anchor；
4. Task 状态两轴模型；
5. 现在工作；
6. 项目简洁视图；
7. Proposal 部分接受；
8. 审计和撤销；
9. JSONL 导出；
10. 基础冲突提示。

MVP 不实现：甘特图、团队权限、实时云同步、复杂日历和完整 Zotero 写入。

文件工作区与外部资源集成规范

本章回答

把脚本、代码、表格、PDF、Git 与 Zotero 纳入项目上下文。

核心结论

笔记负责语义入口，专业系统继续承担各自的权威内容。

1. 目标

项目上下文不仅存在于笔记中，还包括代码、脚本、表格、PDF、图片、压缩包和仓库。系统应提供统一入口和关系，不应把不适合笔记的内容强行复制进 Logseq。

2. 项目文件工作区

Project 或 MiniProject **MAY** 关联一个主文件工作区。推荐结构：

```
project-workspace/  
  README.md  
  inputs/  
  working/  
  outputs/  
  references/  
  archive/  
  manifest.json
```

结构可按项目类型简化，不强制创建空目录。

EXT-WS-001

只有实际需要文件承载时才创建工作区；不得为每个简单 Task 创建目录。

EXT-WS-002

Logseq 保存工作区入口、用途和关键产物说明；文件系统保存实际文件。

3. manifest

manifest 可记录：

- artifact_id;

- path/uri;
- type;
- title;
- related_object_ids;
- role: input/working/output/reference/evidence;
- hash/version;
- created/modified;
- authority provider;
- sensitivity;
- availability。

系统不应假设 manifest 永远完整；可通过扫描提出补全建议。

4. Git 集成

适合代码、配置、文档和可版本化文本。集成应提供：

- 仓库入口；
- 当前分支/提交摘要；
- Output 与 commit/tag 的关系；
- 关联 issue/PR（未来）；
- 工作区脏状态提示。

系统不得自动提交、推送或改分支，除非用户明确授权，并且此类外部副作用独立确认。

5. Zotero 集成

Zotero 作为文献元数据和 PDF 的权威。系统保存 item key/URI、用途、关联对象和阅读状态投影。不得复制管理一套冲突的文献元数据。

Resource 页面可组织多个 Zotero 条目；阅读动作应通过 Task 表达。

6. 文件引用可用性

系统应检测：

- 路径是否存在；
- 移动或重命名；
- 哈希变化；
- 外部磁盘不可用；
- 权限不足；

- 链接失效。

失效时标记 Artifact unavailable，不删除关系。

7. 文件命名与目录

系统 **MAY** 提供推荐命名，但不得因为规范化而大规模重命名既有文件。重命名是高影响外部操作，必须预览受影响引用。

8. Output 登记

当工作完成或文件进入 outputs 时，Agent 可建议注册 Output，包括：

- 产物说明；
- 所属对象；
- 当前版本；
- 是否为交付版；
- 可复用价值；
- 后续维护责任。

文件存在不自动代表正式 Output。

9. 安全

- 不把敏感文件内容默认发送给外部 LLM；
- 只传必要摘要或用户选择内容；
- 外部链接和命令需防止路径注入；
- 删除文件不属于普通语义提交；
- 导出时允许排除敏感路径和元数据。

同步、冲突、备份与恢复规范

本章回答

规定正文、领域状态、锚点和多设备分叉时如何发现、恢复和降级。

核心结论

系统宁可显式冲突，也不能静默覆盖。

1. 可靠性目标

系统必须在插件崩溃、设备切换、正文修改和数据库损坏时避免静默丢失或覆盖。首版优先单用户本地可靠性，不承诺实时多人协作。

2. 一致性单元

一个 SemanticCommit 是跨 Domain Store、Audit Store 和 Logseq 正文的一致性单元。

每次提交前检查：

- 对象 version；
- Anchor content hash；
- Graph identity；
- Proposal base version；
- 外部 Artifact 前置条件（若涉及）。

3. 冲突类型

3.1 正文冲突

Proposal 基于旧正文，当前块已改变。

3.2 领域冲突

对象状态或关系被其他操作修改。

3.3 Anchor 冲突

块缺失、UUID 变化或指向错误内容。

3.4 权威冲突

正文叙事与领域状态明显不一致。

3.5 同步冲突

多个设备分别修改 Domain Store 或 Audit Log。

4. 冲突处理

SYN-CON-001

冲突时 **MUST** 停止可能覆盖用户修改的操作，不得采用“最后写入获胜”静默处理正式正文。

SYN-CON-002

界面必须显示各版本来源、时间和可选处理：保留当前、应用建议、手工合并、重新生成、重新绑定。

SYN-CON-003

Agent 可以提出合并草案，但用户必须确认。

5. 首版同步策略

推荐：

- Logseq 正文使用用户已有 Graph 同步/Git；
- Domain/Audit Store 位于 Graph 可同步的插件数据区或由本地服务管理；
- 单时刻只允许一个写者；
- 打开 Graph 时获取写锁/租约；
- 其他设备在检测到活动租约或版本分叉时进入只读或冲突模式。

该策略牺牲实时多端写入，换取清晰可靠性。

6. 未来同步

可采用事件同步：

- 全局 event_id；
- device_id；
- vector/lamport clock；
- 幂等应用；
- 对可交换低风险事件自动合并；
- 对正文、主归属和类型变化人工冲突解决。

不得简单同步 SQLite 文件并假设无冲突。

7. 备份

至少包含：

- Logseq Graph；
- Domain Store；
- Audit Store；
- Schema/version；
- Skill/规则版本；
- Artifact manifest；
- 可选外部文件清单。

推荐：

- 每日增量；
- 每周完整快照；
- 重大迁移前强制快照；
- 可验证校验和；
- 至少一份独立位置备份。

8. 导出与灾难恢复

系统必须支持生成一个恢复包：

```
export/  
  objects.jsonl  
  relations.jsonl  
  anchors.jsonl  
  events.jsonl  
  commits.jsonl  
  views.json  
  artifacts.jsonl  
  human-readable/  
  schema.json  
  checksums.txt
```

恢复流程：

1. 校验恢复包；
2. 创建空 Domain Store；
3. 导入对象和关系；
4. 重放或校验事件；

5. 连接 Logseq Graph;
6. 验证 Anchor;
7. 生成冲突报告;
8. 重建缓存和视图。

SYN-REC-001

恢复演练必须成为发布验收的一部分，不能只存在文档中。

9. Schema 迁移

每次 schema 迁移必须：

- 前置备份；
- dry-run 报告；
- 可重入；
- 记录迁移事件；
- 提供回滚或恢复方案；
- 对不可逆变更显式告知。

10. 故障降级

- Agent 失败：继续手工使用；
- Logseq Adapter 失败：领域层只读，提供导出；
- Domain Store 失败：禁止继续状态写入，正文仍可普通编辑；
- Audit Store 失败：禁止正式 SemanticCommit；
- 外部 Artifact 不可用：保留引用并提示。

PART V

场景、验证与治理

用真实工作证明规范可执行，并让未来演进保持可追溯。

规范只有能够映射到场景、实现与测试时，才真正成为约束。

核心使用场景目录

本章回答

用二十个真实工作场景串联多份规范。

核心结论

场景证明规则可以连续工作，而不是各自正确却无法组合。

场景回放：接受文字改写，但拒绝移动位置

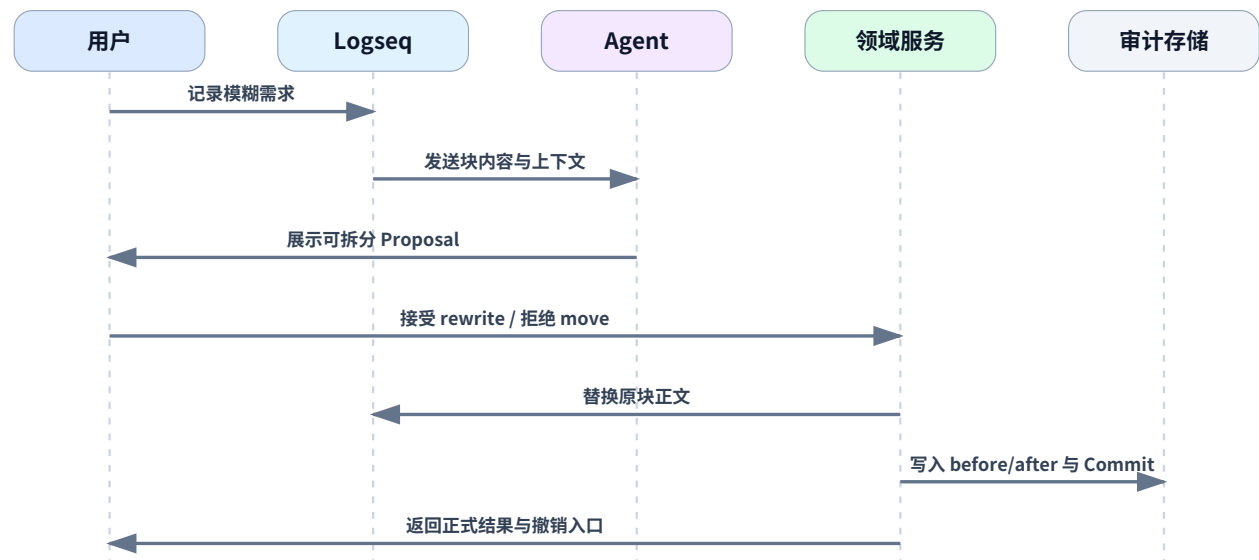


图 11 部分接受场景贯穿正文、领域服务和审计存储。

1. 用途

场景用于验证多份规范能否在真实工作中连续工作。每个场景必须明确入口、用户目标、系统行为、审查点、结果和涉及规则。

UC-01：随手记录一段模糊需求

输入：

下午有人说可能需要把听云和 Graylog 的告警关联起来，具体怎么做还没定。

期望：

- 原始内容低摩擦保存为 Capture；

- 不直接创建 Ready Task；
- Agent 提取“存在潜在需求”和“方案/目标不明确”；
- 建议保留为待澄清事项或关联现有调研；
- 不生成空的背景、风险和步骤结构。

关键规则： PRI-003、SEM-CAP-001、CAP-TRI-001、TXT-STRUCT-002。

UC-02：明确需求正式化为 MiniProject

输入：

下周前需要完成 Graylog 告警到外部推送的整体链路说明，现有脚本可以继续用，但还要找真实事件验证。

期望：

- Agent 建议 MiniProject；
- 提供目标、边界、完成判据；
- 生成静态梳理和真实验证两个候选 Task；
- 文字、类型、归属、位置和 Task 分别审查。

UC-03：接受改写，拒绝移动位置

操作： 用户接受 rewrite_content 和 create_object，拒绝 move_content。

期望：

- Journal 原块被正式文字替换；
- 原始版本进入 Audit Store；
- 对象通过 Anchor 与原块绑定；
- 主归属可独立确认；
- 内容不被移动。

关键规则： PRI-004、PRI-006、REL-PLC-002、REV-PART-002、MAP-RWT-001。

UC-04: 一段记录更新现有 Task

输入:

今天已经核对完 Event Definition 和 Notification 的对应关系，但外部推送还没真实验证。

期望:

- 匹配现有 Task/MiniProject;
- 建议完成静态核对 Task;
- 更新当前状态摘要;
- 建议新的下一步，但不重复创建已有对象;
- 事件流保留原记录。

UC-05: Task 进入 Waiting

输入:

已经把接口问题发给厂商，等他们确认支持范围，周五再跟进。

期望:

- Phase 保持 READY/ACTIVE;
- Condition 变为 WAITING;
- waiting_for=厂商;
- 期待结果=确认接口范围;
- review_at=周五;
- 到期生成 REVIEW_DUE Signal。

UC-06: Blocked 与 Waiting 区分

输入 A: 已提交权限申请，等待管理员审批。

应为 WAITING。

输入 B: 当前不知道需要哪类权限，也没有明确处理人。

应为 BLOCKED 或 CLARIFY，并建议确定权限和责任人。

UC-07：简单内容不生成空结构

输入：

明天重启 10.195.1.83 上的 zabbix-agent2，并确认服务恢复。

期望：一个 Task，自然语言描述和必要日期即可；不生成 Project 风险、里程碑、成果等栏目。

UC-08：MiniProject 升级建议

现状：一个 MiniProject 已形成独立目标、三个子 MiniProject、独立文件仓库和长期利益相关人。

期望：Agent 引用 MIG-UP 规则提出 Project 升级建议，同时列出反对信号；未经确认不执行。

UC-09：Project 中断三周后重入

操作：用户请求恢复上下文。

期望：重入包回答目标、边界、阶段、最近变化、有效结论、旧信息失效、阻塞、恢复动作和三个以内关键入口；不得仅输出长篇历史摘要。

UC-10：Project 可能已完成

现状：目标结果已交付，仍有一个“整理经验”Task 和未登记脚本。

期望：系统发出可能进入 CLOSING 和 UNHARVESTED_OUTPUT 信号；不自动完成 Project；提供收尾检查。

UC-11：归属与位置冲突

现状：领域主归属为 Graylog Project，正文位于“运维智能体”页面。

期望：不自动移动；提示位置与主上下文可能不一致，提供保持位置、移动或修改归属三个独立选择。

UC-12：同一输入涉及多个项目

期望：选择一个主归属，建立具体跨项目关联；不得创建两个独立可编辑副本。

UC-13：正文在 Proposal 生成后被用户修改

期望：提交时发现 hash/version 冲突，停止覆盖；显示当前正文和旧建议，允许重生成或手工合并。

UC-14：Agent 推断错误

输入：“可能跟负载均衡有关。”

期望：正式文本保持“可能”，不得写成“问题由负载均衡导致”；归属和技术结论需要待确认。

UC-15：批量 Inbox 清理

期望：可批量把明显普通笔记标为无需管理；涉及主归属、类型迁移和项目完成的 Proposal 必须逐项审查。

UC-16：文件成果登记

现状：项目工作区新增脚本和说明文档。

期望：Agent 建议将其登记为 Output，说明版本和用途；不得仅因文件存在自动认定为最终交付。

UC-17：Anchor 失联

现状：用户删除主正文块。

期望：对象保留，Anchor 标 missing，系统提供重新绑定、无正文保留或归档；不得删除对象。

UC-18: 插件或 Agent 不可用

期望: Logseq 正文可读; 用户可手动查看和更新基础事务; 导出和恢复可用; 不产生散乱备用属性。

UC-19: 周度审查

期望: 按问题聚合 NoNextAction、ReviewDue、Stale、BoundaryDrift、未登记成果和长期 Capture; 不是把所有项目完整列一遍。

UC-20: 更换 LLM

期望: 新模型生成不同 Proposal 不影响已确认对象、状态和历史; Proposal 记录模型和 Skill 版本, 可运行黄金样例回归。

黄金样例与反例库

本章回答

提供内容改写、对象分类、状态判断与结构克制的正反样例。

核心结论

黄金样例把抽象规范转化为 Agent 可以学习和回归的行为。

1. 用途

黄金样例用于训练、评估和回归 Agent 行为。样例不是机械模板；它们展示事实保真、粒度判断、结构克制和语义操作拆分。

EX-01：口语需求的忠实改写

原始内容

下午领导说下周前最好把 Graylog 告警到外部推送这部分梳理清楚，之前脚本应该还能用，但最好再找个真实事件跑一下。

合格改写

下午收到新的工作要求：需要在下周前完成 Graylog 告警生成、中间处理和外部推送链路的整体梳理。现有只读审计脚本可以继续作为基础，但还需要选择一条真实事件验证完整链路。

为什么合格

- 修复口语但未改变事实；
- 保留“下周前”“现有脚本”“真实验证”；
- 没有杜撰负责人、具体日期或技术结论；
- 用自然段表达，而不是属性罗列。

不合格

背景：Graylog 告警体系不完善。**目标：**全面优化告警能力。**风险：**推送失败。**下一步：**完成 5 个阶段的系统改造。

问题：杜撰现状、夸大目标、生成空泛结构。

EX-02: Task 与 MiniProject

输入 A

把 Graylog 推送脚本的 token 改为新值并执行一次测试。

判断：Task。存在单一动作和完成标准。

输入 B

梳理 Graylog 告警从日志进入 Stream、触发 Event、执行 Notification 到外部接收的完整链路，并形成说明和验证证据。

判断：MiniProject。需要多个步骤、局部上下文和成果。

反例

因为 B 只有一句话就判断为 Task，或因为 A 可能涉及多个命令就判断为 MiniProject，均错误。

EX-03: Project 与 Area

Project

在 2026 年完成 RHCSA 考试准备并通过考试。

有明确终态、时间和成果。

Area

持续维护 Linux 系统运维能力。

没有天然完成时刻，需要维持标准。

边界

“RHCSA/RHCE 学习”可以作为 Area；其中“通过 RHCSA”与“通过 RHCE”分别是 Project。

EX-04: Waiting 与 Blocked

Waiting

已把接口清单发给厂商，等待确认哪些 API 可用，7 月 20 日复查。

Blocked

无法继续调查，因为目前不知道生产平台的认证方式，也没有确定咨询对象。

前者知道外部责任和期待结果；后者需要先解除未知阻碍。

EX-05: 位置与归属独立

原文位于 Journal：

今天完成了法务系统 8:45 事件的 Trace 和 Call Tree 核查。

合格处理：原地改写并关联“听云 APM 调研”Project，建立 progress event。没有必要移动到项目页。

不合格处理：为了归属直接剪切原文，导致当天工作时间线缺失。

EX-06：部分接受

Proposal:

- 1. 改写正文；
- 2. 创建 MiniProject；
- 3. 归入 Graylog Project；
- 4. 移动到项目页；
- 5. 创建两个 Task。

用户结果：接受 1、2、其中一个 Task；拒绝 3、4、另一个 Task。

合格提交：正文原地替换；MiniProject 创建并标 UNASSIGNED；一个 Task 创建；其他不执行。

不合格提交：因归属被拒绝而取消全部操作，或偷偷使用页面位置作为归属。

EX-07：项目当前状态

合格

当前已经完成 Event Definition、Notification 与外部推送脚本之间的静态关系梳理，但尚未通过真实事件验证完整链路。下一步需要选择一条非 Test 事件，确认从日志进入 Stream 到外部平台接收消息的闭环。

不合格：属性堆叠

状态：进行中；已完成：静态关系；未完成：真实验证；下一步：选择事件；风险：未知。

不合格：历史倾倒

把十次调查日志完整复制到项目首页。

EX-08：项目重入包

合格结构

- 一段项目定位；

- 一段当前状态；
- 最近三项关键变化；
- 两个未决问题；
- 一个建议恢复动作；
- 三个以内关键入口。

不合格

按日期压缩过去全部记录，但没有说明现在该做什么。

EX-09：事实、推断和建议

原文：

海丝那边说路由还是有问题，可能和负载均衡有关。

合格：

- 事实：对方反馈路由仍有问题；
- 推断：可能涉及负载均衡，尚未确认；
- 建议：先核对原有策略和实际访问链路。

不合格：

已确认负载均衡配置导致路由故障。

EX-10：结构按需出现

简单 Task

明天下午重启 datanode15 的 zabbix-agent2，并确认 10050 端口恢复监听。

无需背景、风险、里程碑栏目。

Project

“任务管理系统插件”需要目标、边界、阶段、架构决策和成果入口，但无有效风险时仍不显示风险栏目。

EX-11：迁移依据

合格建议：

建议将“信息中心事件聚合”从 MiniProject 升级为 Project。它已经形成独立目标、单独仓库、多个子工作流和持续维护计划，即使原上层项目结束仍需继续。依据 MIG-UP-001/002。

不合格：

内容已经很多，建议升级为 Project。

EX-12：Agent 不应过度介入

用户只写了一段普通技术笔记，没有行动或归属需求。

合格：不主动生成大型 Proposal；可以保持普通笔记。

不合格：自动创建 Task、Resource、Decision 和五个标签。

验收标准、测试策略与追踪矩阵

本章回答

定义 MVP 验收、测试层级、质量指标和规则追踪矩阵。

核心结论

每条 MUST 规则都应能追踪到实现、Skill、界面或测试。

推荐演进路径：先验证语义，再扩大自动化与前端范围



每一阶段都必须先通过场景回放与恢复演练，再进入下一阶段。

图 12 从语义原型到跨前端集成的演进路径。

1. 验收目标

系统不仅要“功能点”，还必须证明：语义一致、正式修改可控、正文可读、故障可恢复、Agent 不越权。

2. MVP 功能验收

TST-MVP-001

捕获

从当前 Logseq 块创建 Capture，不要求额外字段，原始内容和来源完整保存。

TST-MVP-002

独立对象

创建 Task/MiniProject 后，状态存于 Domain Store，正文不出现大量属性。

TST-MVP-003

Anchor

块移动后对象仍可打开；块删除后对象不被删除并产生冲突。

TST-MVP-004

两轴状态

Task 可处于 ACTIVE + WAITING，并具有 waiting_for/review_at；Waiting 到期生成 Signal。

TST-MVP-005

部分接受

可以接受 rewrite、拒绝 move 和 ownership，提交结果符合依赖规则。

TST-MVP-006

审计和撤销

正文改写和状态变化可查看 before/after，并可撤销。

TST-MVP-007

现在工作

视图仅显示 Actionable 和高价值注意项，不展示完整历史。

TST-MVP-008

重入

对样例 Project 生成符合 VIEW-RE-001 的重入包。

TST-MVP-009

导出恢复

从恢复包重建对象、关系、事件和 Anchor 报告。

TST-MVP-010

无 Agent 降级

关闭 LLM 后基础捕获、对象编辑、状态流转和视图仍可使用。

3. 测试层级

3.1 单元测试

- 状态转换矩阵；
- 主归属唯一；

- 依赖循环；
- Signal 计算；
- 操作依赖图；
- View DSL；
- Schema 校验。

3.2 领域契约测试

对所有 Adapter 和前端复用：

- Command 输入输出；
- 版本冲突；
- 错误码和 rule_refs；
- 事件原子写入；
- 回滚。

3.3 Adapter 测试

- Block CRUD；
- 页面重命名；
- 块移动/删除；
- File Graph/DB Graph；
- Unicode 和长文本；
- 插件重启。

3.4 Agent 黄金测试

使用 `23-黄金样例与反例库.md`：

- 分类准确；
- 事实保真；
- 杜撰率；
- 规则引用；
- 结构克制；
- 操作拆分；
- 语言质量；
- 多次运行稳定性。

3.5 场景测试

自动或半自动执行 `22-核心使用场景目录.md`，检查端到端结果。

3.6 故障与恢复测试

- Commit 中途失败；
- Audit Store 不可写；
- 正文版本冲突；
- 数据库损坏恢复；
- Anchor 失联；
- 两设备版本分叉；
- Schema 升级回滚。

4. 质量指标

4.1 体验指标

- 捕获典型输入所需动作数；
- 从项目中断到识别下一步的时间；
- 工作台无关信息比例；
- 每周手工维护属性数量；
- Proposal 平均审查时间；
- 用户撤销率。

4.2 Agent 指标

- 事实保真率；
- 归属 Top-1/Top-3 准确率；
- 对象类型准确率；
- 不必要结构生成率；
- 杜撰率；
- 高影响建议误触发率；
- 部分接受后合法提交率。

4.3 系统指标

- Commit 原子成功率；
- Anchor 有效率；
- 恢复演练成功率；
- 导出完整率；
- 视图响应时间；
- 数据一致性异常数。

5. 发布门槛

首个个人可用版本必须：

- 所有 **MUST** 规则对应至少一个测试或人工检查；
- High 风险操作无自动越权；
- 100 次部分接受随机组合不产生非法状态；
- 备份恢复演练通过；
- 黄金样例无事实杜撰；
- Logseq 正文在插件关闭时仍可读；
- 无已知静默数据覆盖问题。

6. 追踪矩阵（核心节选）

规则	实现约束	UI/Skill	测试
PRI-003 渐进正式化	Capture 独立表	Inbox/分诊 Skill	UC-01、TST-MVP-001
PRI-004 正文唯一历史保留	文本版本事件	Diff/撤销	UC-03、TST-MVP-006
PRI-006 建议可拆分	SemanticOperation DAG	分项审查	UC-03、UC-15、TST-MVP-005
REL-OWN-001 单主归属	DB 唯一约束	Ownership Skill	多主归属拒绝测试
REL-PLC-002 移动独立	move_content 独立命令	Review UI	UC-03、UC-11
LIF-BASE-001 三轴分离	phase/condition/signal	状态组件	UC-05、UC-06
INF-OWN-001 单类单权威	authority registry	冲突提示	UC-11、UC-13
MAP-ANC-001 稳定 ID	object_id + Anchor	插件适配	UC-17、TST-MVP-003
AUD-EVT-001 原子审计	transaction/saga	历史视图	故障注入测试
COM-ATM-001 原子提交	Commit executor	提交结果	随机组合/失败补偿
VIEW-RE-001 行动导向重入	Reentry query	Reentry Skill	UC-09、TST-MVP-008
TXT-STRUCT-002 无空标题	renderer/Skill lint	正文生成	EX-01、EX-10
AGT-AUTH-001 领域命令	API 权限	Agent tools	越权测试
SYN-REC-001 恢复演练	export/import	恢复 UI	TST-MVP-009

7. 规范覆盖审计

每次发布自动生成：

- 所有规则 ID 列表；
- 是否被代码、Skill、测试引用；
- 未覆盖 **MUST** 规则；
- 已废弃规则；
- 文档断链；
- ADR 变更影响。

未覆盖 **MUST** 规则必须阻止正式发布，除非 ADR 明确降级或延期。

ADR 模板与规范治理流程

本章回答

规定重要决策、规范变更、废弃和版本治理流程。

核心结论

未来改变设计时，必须知道改了什么、为何改变、如何验证。

1. 目的

系统需求会变化，但变化必须可解释。ADR（Architecture/Domain Decision Record）记录重要决定的背景、选择、代价和重审条件，防止规则在实现中悄然漂移。

2. 需要 ADR 的变化

- 新增或删除一等对象类型；
- 修改 Task/MiniProject/Project/Area 边界；
- 修改状态机或主归属规则；
- 改变信息权威来源；
- 允许新的 Agent 自动执行权限；
- 改变正式内容替换和历史策略；
- 更换 Domain Store/同步架构；
- 引入不可逆外部操作；
- 破坏现有导出或 Adapter 兼容性；
- 偏离不可变原则的阶段性疾病。

3. ADR 模板

```
# ADR-XXXX: 标题

- 状态: proposed / accepted / superseded / rejected
- 日期: YYYY-MM-DD
- 决策者:
- 影响规范:
- 影响规则 ID:

## 背景

为什么现在需要作出决定？当前痛点、证据和约束是什么？

## 决策驱动因素
```

按优先级列出，例如数据安全、捕获摩擦、可移植性、实现成本。

候选方案

方案 A

描述、优点、缺点、风险。

方案 B

.....

决策

选择什么，以及明确不选择什么。

规则变化

新增、修改、废弃哪些规则；迁移期如何处理。

后果

正面、负面、技术债、运维影响、用户体验变化。

验证

需要哪些场景、黄金样例、测试和指标证明决定有效。

迁移与回滚

数据、Skill、UI、Adapter 如何升级；何时可以回滚。

重审条件

出现什么证据时应重新讨论。

4. 规范变更流程

1. 提出问题和证据；
2. 判断是否需要 ADR；
3. 标识受影响规则和文档；
4. 更新领域规范草案；
5. 更新场景、黄金样例和测试；
6. 评审是否违反上层原则；
7. 接受 ADR；
8. 实现与迁移；
9. 运行追踪矩阵和回归；
10. 发布新规范版本。

GOV-001

不得先修改代码或 Skill，再事后让规范追认重大语义变化。

5. 文档唯一归属

规则应只在一份权威文档中定义：

- 对象定义：03；
- 关系：04；
- 生命周期：05；
- 迁移：06；
- 权威来源：07；
- Anchor：08；
- 审计：09；
- 捕获：10；
- 提交：11；
- 审查：12；
- 视图：13-15；
- Agent/Skill：16-17；
- 技术实现：18-21；
- 验证：22-24。

其他文档只能引用，不得复制一份略有差异的规则。

6. 版本策略

- Patch：澄清文字，不改变行为；
- Minor：新增兼容规则、视图或操作；
- Major：对象语义、权威来源、提交模型或兼容性发生破坏性变化。

每个 Proposal、SemanticCommit 和 DomainEvent 应记录适用的规范版本。

7. 废弃流程

规则废弃必须：

- 标记 deprecated；
- 指向替代规则；

- 说明迁移期限；
- 更新追踪矩阵；
- 保留历史版本供旧事件解释。

8. 定期审查

建议每个重要原型阶段后审查：

- 哪些规则在真实使用中造成过高摩擦；
- 哪些概念边界仍经常误判；
- 哪些 Agent 建议经常被拒绝；
- 哪些视图没有帮助行动；
- 哪些数据重复成为双主；
- 是否出现新对象需求；
- 是否有规则无法测试。

审查结果通过 ADR 进入规范，而不是直接在实现中加例外。

附录：稳定规则索引

本索引用于从规则编号快速定位权威定义。正文中的规则卡片是完整解释；此处只列出标题与权威文档。

规则编号	规则标题	权威文档
AGT-AUTH-001	Agent 不直接获得数据库任意写权限；必须通过领域命令/API。	16-Agent角色权限与自动化边界
AGT-TRUTH-001	Inferred/Suggested 不得在正文中使用确定语气，除非用户确认后成为正式判断。	16-Agent角色权限与自动化边界
ARC-CACHE-001	缓存失效不得导致事实丢失；最坏情况是重新生成或暂时不显示。	18-总体系统架构与领域数据模型
ARC-CQRS-001	Agent 不得使用底层 SQL 直接修改状态，必须调用命令。	18-总体系统架构与领域数据模型
ARC-LAYER-001	Interaction 和 Adapter 层不得实现独立的对象语义或状态机。	18-总体系统架构与领域数据模型
ARC-LAYER-002	Domain Layer 不得依赖 Logseq API；只能依赖抽象 Anchor 和正文读写接口。	18-总体系统架构与领域数据模型
AUD-EVT-001	事件写入必须与领域状态提交原子一致：状态成功而事件失败、或事件成功而状态失败均不合法。	09-版本审计与回滚规范
AUD-RBK-001	回滚本身也是新事件，不得改写或删除旧事件。	09-版本审计与回滚规范
AUD-TXT-001	正式内容替换后，原始文本不必在正常正文并列显示，但 MUST 能从审查记录或历史视图恢复。	09-版本审计与回滚规范
CAP-FRM-001	正式内容替换原始正文并不自动等于 Capture 成为新对象；它也可能只是现有对象的进展或普通正式笔记。	10-捕获分诊与正式化流程规范
CAP-IN-001	捕获时 MUST NOT 强制用户填写对象类型、归属、优先级、截止时间和完整模板。	10-捕获分诊与正式化流程规范
CAP-IN-002	系统必须保存原始文本、来源位置、时间和捕获方式。	10-捕获分诊与正式化流程规范
CAP-TRI-001	分诊结果必须包含依据和不确定项。	10-捕获分诊与正式化流程规范
CAP-TRI-002	同一 Capture 可以同时更新现有对象并产生新 Task，但每项操作独立审查。	10-捕获分诊与正式化流程规范
COM-ATM-001	一次 Commit 必须表现为“全部已确认操作成功”或“没有任何正式变化”。若跨系统无法实现真实事务，必须采用可恢复的 saga 并向用户暴露状态。	11-Agent语义建议与提交协议
COM-OP-001	文字改写、对象分类、主归属、物理位置、关系和状态必须是独立操作。	11-Agent语义建议与提交协议
COM-OP-002	操作之间若存在语义依赖必须显式表达。例如创建 Task 可能依赖 create_object，但不必依赖 move_content。	11-Agent语义建议与提交协议
COM-PROP-001	Proposal 是建议，不得被查询视图当作正式事实。	

规则编号	规则标题	权威文档
		11-Agent语义建议与提交协议
DISC-001	查看越轻，界面越靠近原文；决策越复杂，界面越独立。	15-结构显隐与渐进披露规范
DISC-002	无有效风险时不显示风险栏目；无 Waiting 时不显示等待区域；无 Decision 时不显示决策标题。	15-结构显隐与渐进披露规范
DISC-003	结构的存在由数据判断，显示由视图判断，两者不得混为一体。	15-结构显隐与渐进披露规范
EXT-WS-001	只有实际需要文件承载时才创建工作区；不得为每个简单 Task 创建目录。	20-文件工作区与外部资源集成规范
EXT-WS-002	Logseq 保存工作区入口、用途和关键产物说明；文件系统保存实际文件。	20-文件工作区与外部资源集成规范
GOV-001	不得先修改代码或 Skill，再事后让规范追认重大语义变化。	25-ADR模板与规范治理流程
INF-EVT-001	过程性进展以事件形式持续增长；当前摘要只表达仍然有效的结论和状态。	07-信息分层与权威来源规范
INF-EVT-002	Agent 更新当前摘要时必须引用导致变化的事件，不得凭空重写。	07-信息分层与权威来源规范
INF-EVT-003	旧判断被推翻时，应在当前摘要中移除或标记失效，但历史事件不得删除。	07-信息分层与权威来源规范
INF-OWN-001	每个字段或内容片段必须声明 authority class。系统不得允许两个位置同时作为可写权威。	07-信息分层与权威来源规范
INF-OWN-002	Logseq 正文变化可触发领域建议，但不得仅凭自然语言自动改变高影响领域状态。	07-信息分层与权威来源规范
INF-OWN-003	Domain Store 可以把状态投影为徽标、悬浮窗或查询块，但这些投影不得成为独立编辑副本。	07-信息分层与权威来源规范
INF-OWN-004	生成视图可随时删除和重建。任何无法重建的关键事实不得只存在于视图缓存。	07-信息分层与权威来源规范
LIF-BASE-001	Phase、Condition 和 Signal MUST 独立存储和计算，不得互相代替。	05-生命周期与状态机规范
LIF-COND-001	WAITING 必须具有 `waiting_for`、期待结果和 `review_at`。	05-生命周期与状态机规范
LIF-COND-002	BLOCKED 必须具有阻塞说明；SHOULD 关联 blocker 或待澄清动作。	05-生命周期与状态机规范
LIF-COND-003	PAUSED 必须记录暂停理由和可选复查时间。	05-生命周期与状态机规范
LIF-PROJ-001	Project 进入 PLANNED 前必须具有目标、边界、完成判据和责任 Area 或明确例外。	05-生命周期与状态机规范
LIF-PROJ-002	Project 进入 ACTIVE 前必须存在当前推进或可执行下层对象。	05-生命周期与状态机规范
LIF-PROJ-003		

规则编号	规则标题	权威文档
	Project 进入 COMPLETED 前必须检查：目标结果、未完成下层对象、未决等待、成果登记、归档入口和复盘需要。	05-生命周期与状态机规范
LIF-SIG-001	Signal MUST 可根据事实重新计算，不得由用户手工当作永久标签维护。	05-生命周期与状态机规范
LIF-TASK-001	Task 进入 READY 前必须满足 `SEM-TASK-001`。	05-生命周期与状态机规范
LIF-TASK-002	Condition WAITING/BLOCKED/PAUSED 可与 READY 或 ACTIVE 组合，而不是取代 Phase。	05-生命周期与状态机规范
LIF-TASK-003	Task 完成必须记录完成事件；若存在 Output，应建立 produces 关系。	05-生命周期与状态机规范
MAP-ANC-001	对象身份必须来自 `object_id`，不得来自页面标题、块路径或正文文本哈希。	08-Logseq映射与对象锚点规范
MAP-ANC-002	一个对象最多拥有一个 active `primary_text` Anchor，但可拥有多个 source/context/event Anchor。	08-Logseq映射与对象锚点规范
MAP-PAGE-001	重命名页面不得改变 Project 身份。	08-Logseq映射与对象锚点规范
MAP-PAGE-002	删除项目页面不等于删除 Project；应产生 missing anchor 冲突并提供重建入口。	08-Logseq映射与对象锚点规范
MAP-RWT-001	仅文字替换不得改变对象类型、主归属和物理位置。	08-Logseq映射与对象锚点规范
MIG-001	身份连续性	06-对象迁移拆分合并规范
MIG-002	历史不可重写	06-对象迁移拆分合并规范
MIG-003	建议与执行分离	06-对象迁移拆分合并规范
MIG-004	迁移不等于移动	06-对象迁移拆分合并规范
MIG-MERGE-001	相同主题不等于重复目标。两个都与 Graylog 有关的 Project 不应因此合并。	06-对象迁移拆分合并规范
MIG-UP-001	“内容很多” 不能作为唯一升级依据。	06-对象迁移拆分合并规范
MIG-UP-002	升级为 Project 后必须重新确定责任 Area、目标终态和完成判据。	06-对象迁移拆分合并规范
PLG-UI-001	插件不得在正文中永久插入完整属性面板。	19-Logseq插件与适配层设计
PLG-UI-002	关闭插件后，正式正文仍应保持人类可读；任务状态可能无法完整显示，但不得只剩不可理解的 ID。	19-Logseq插件与适配层设计
PLG-UI-003	所有写入前显示最终影响；提交后显示结果和撤销。	19-Logseq插件与适配层设计
PRI-001	优化认知成本，而不是结构数量	

规则编号	规则标题	权威文档
		01-项目愿景边界与不可变原则
PRI-002	自然语言优先	01-项目愿景边界与不可变原则
PRI-003	渐进式正式化	01-项目愿景边界与不可变原则
PRI-004	正文唯一，历史可追溯	01-项目愿景边界与不可变原则
PRI-005	用户保持最终控制权	01-项目愿景边界与不可变原则
PRI-006	建议可拆分	01-项目愿景边界与不可变原则
PRI-007	位置不等于语义	01-项目愿景边界与不可变原则
PRI-008	完整数据与克制视图分离	01-项目愿景边界与不可变原则
PRI-009	单类信息单一权威	01-项目愿景边界与不可变原则
PRI-010	确定性规则优先	01-项目愿景边界与不可变原则
PRI-011	Agent 解释而非伪装确定	01-项目愿景边界与不可变原则
PRI-012	核心语义独立于前端	01-项目愿景边界与不可变原则
PRI-013	默认可逆	01-项目愿景边界与不可变原则
PRI-014	事件与当前状态分离	01-项目愿景边界与不可变原则
PRI-015	低打扰治理	01-项目愿景边界与不可变原则
REL-DEP-001	`depends_on` 必须具有方向，且领域层禁止循环依赖。	04-关系归属与层级规范
REL-DEP-002	只有当前对象确实不能达到某个阶段或完成条件时才建立依赖；普通相关性不得伪装为依赖。	04-关系归属与层级规范
REL-DEP-003	Waiting 不一定需要依赖对象，但必须记录等待主体/事件、期待结果和复查时间。	04-关系归属与层级规范
REL-DEP-004	Blocked SHOULD 尽量关联具体 blocker；若未知，必须记录阻塞说明并进入待澄清。	04-关系归属与层级规范
REL-GRAPH-001	视图聚合不得改变主归属。	04-关系归属与层级规范
REL-GRAPH-002	对象可出现在任意数量的视图中，但状态和内容不得复制为多个可独立编辑副本。	04-关系归属与层级规范
REL-OUT-001	Project 完成前 SHOULD 检查是否存在已经形成但未登记的 Output。	04-关系归属与层级规范
REL-OWN-001	一个工作对象在同一时间最多拥有一个直接主归属。	04-关系归属与层级规范

规则编号	规则标题	权威文档
REL-OWN-002	主归属表达责任和主要解释上下文，而不是物理存放位置。	04-关系归属与层级规范
REL-OWN-003	Task 可以直接归属于 MiniProject、Project 或 Area。系统不得为了形式完整而强制建立空 MiniProject。	04-关系归属与层级规范
REL-OWN-004	MiniProject 通常归属于 Project，也可以直接归属于 Area；若直接归 Area，应说明为什么无需独立 Project。	04-关系归属与层级规范
REL-OWN-005	Project 通常归属于 Area，也可以暂时处于 Unassigned/Incubating 区域，但 Active Project SHOULD 明确责任 Area。	04-关系归属与层级规范
REL-OWN-006	Area 不得相互形成无限层级。首版最多允许两层 Area，且子 Area 必须有清楚的责任边界。	04-关系归属与层级规范
REL-PLC-001	Logseq 中正文块可以留在 Journal、项目页或主题页，位置本身不构成主归属。	04-关系归属与层级规范
REL-PLC-002	移动块是独立语义操作，必须与文字改写和归属修改分别审查。	04-关系归属与层级规范
REL-PLC-003	若内容留在 Journal，但已正式关联 Project，Project 视图应通过引用或投影显示必要信息，不得要求复制原文。	04-关系归属与层级规范
REL-PLC-004	物理移动不得破坏来源时间和原始上下文。系统必须保留 `sourced_from` 锚点。	04-关系归属与层级规范
REL-SRC-001	每个正式化自 Capture 的对象或正文变更必须保留来源锚点。	04-关系归属与层级规范
REL-SRC-002	来源表达“从哪里来”；上下文表达“理解时有帮助”。两者不得混用。	04-关系归属与层级规范
REL-SRC-003	同一个来源可以产生多个对象；同一个对象也可以有多个来源。	04-关系归属与层级规范
REV-FAT-001	系统不得为没有实质变化的内容生成 Proposal。	12-审查与部分接受规则
REV-FAT-002	纯语言改写若不能显著改善准确性、可读性或后续管理，不应打扰用户。	12-审查与部分接受规则
REV-FAT-003	系统 SHOULD 学习用户反复拒绝的建议类型，但学习结果只影响推荐和默认选择，不得绕过正式规则。	12-审查与部分接受规则
REV-PART-001	每个 SemanticOperation 必须可独立接受、编辑、拒绝或暂缓，除非其契约声明强依赖。	12-审查与部分接受规则
REV-PART-002	内容修改与位置调整必须分别审查。	12-审查与部分接受规则
REV-PART-003	内容修改与主归属必须分别审查。	12-审查与部分接受规则
REV-PART-004	创建父对象不意味着必须接受 Agent 生成的所有子任务。	12-审查与部分接受规则
REV-PART-005	用户编辑 Agent 建议后，最终内容视为用户确认版本，审计中同时保留 Agent 原建议。	12-审查与部分接受规则
REV-UI-001	界面不得只提供“全部应用”而隐藏语义操作。	12-审查与部分接受规则
SEM-AREA-001	“学习 Linux”若表示持续能力建设，属于 Area；“通过 2026 年 RHCSA 考试”属于 Project。	03-事务对象语义与契约规范
SEM-CAP-001	Capture 只是输入状态，不得被默认当作 Task。	03-事务对象语义与契约规范
SEM-COMMON-001	对象类型 MUST 由语义行为决定，不得只按文本长度、子块数量或标题形式判断。	03-事务对象语义与契约规范
SEM-COMMON-002	不存在有效内容的可选结构 MUST NOT 为了模板完整而生成。	03-事务对象语义与契约规范
SEM-COMMON-003		

规则编号	规则标题	权威文档
	正文内容不足以满足对象最低契约时，对象 MUST 处于 Clarify/Defining，而不得伪装为 Ready/Active。	03-事务对象语义与契约规范
SEM-DEC-001	Agent 推断不得直接创建 accepted Decision。只有用户明确确认或已有权威证据时才能接受。	03-事务对象语义与契约规范
SEM-MINI-001	MiniProject 的“Mini”由治理独立性和边界决定，不由任务数量决定。	03-事务对象语义与契约规范
SEM-MINI-002	一个只有三行内容但拥有独立目标和边界的工作，可能是 Project；一个拥有十个简单步骤但完全服务于单一有限结果的工作，仍可能是 MiniProject。	03-事务对象语义与契约规范
SEM-PROJ-001	Project 的核心不是任务集合，而是目标、边界、阶段和成果的独立治理单元。	03-事务对象语义与契约规范
SEM-PROJ-002	Project 若长期无法定义结束条件，系统 SHOULD 建议其转为 Area、Resource 或拆分为可结束 Project。	03-事务对象语义与契约规范
SEM-RES-001	Resource 不应被强制纳入执行状态机。若阅读资源需要行动，应另建 Task 并关联 Resource。	03-事务对象语义与契约规范
SEM-TASK-001	Task MUST 具有可判断的完成结果。“研究一下”“持续关注”若没有明确终态，不能直接成为 Ready Task。	03-事务对象语义与契约规范
SEM-TASK-002	Task 的背景 SHOULD 足以避免未来重新理解，但 MUST NOT 复制完整项目上下文。	03-事务对象语义与契约规范
SKL-001	Skill 输出必须结构化，不能只返回一段无法审计的自然语言。	17-Skill架构与编写规范
SKL-002	Skill 不得复制并修改上游规则；应引用规则 ID。必要的摘要必须声明仅为解释。	17-Skill架构与编写规范
SYN-CON-001	冲突时 MUST 停止可能覆盖用户修改的操作，不得采用“最后写入获胜”静默处理正式正文。	21-同步冲突备份恢复规范
SYN-CON-002	界面必须显示各版本来源、时间和可选处理：保留当前、应用建议、手工合并、重新生成、重新绑定。	21-同步冲突备份恢复规范
SYN-CON-003	Agent 可以提出合并草案，但用户必须确认。	21-同步冲突备份恢复规范
SYN-REC-001	恢复演练必须成为发布验收的一部分，不能只存在文档中。	21-同步冲突备份恢复规范
TST-MVP-001	捕获	24-验收标准测试策略与追踪矩阵
TST-MVP-002	独立对象	24-验收标准测试策略与追踪矩阵
TST-MVP-003	Anchor	24-验收标准测试策略与追踪矩阵
TST-MVP-004	两轴状态	24-验收标准测试策略与追踪矩阵
TST-MVP-005	部分接受	24-验收标准测试策略与追踪矩阵
TST-MVP-006	审计和撤销	24-验收标准测试策略与追踪矩阵

规则编号	规则标题	权威文档
TST-MVP-007	现在工作	24-验收标准测试策略与追踪矩阵
TST-MVP-008	重入	24-验收标准测试策略与追踪矩阵
TST-MVP-009	导出恢复	24-验收标准测试策略与追踪矩阵
TST-MVP-010	无 Agent 降级	24-验收标准测试策略与追踪矩阵
TXT-DEN-001	系统不得在 D1/D2 视图中直接倾倒 D3/D4 内容。	14-自然语言呈现与信息密度规范
TXT-STRUCT-001	只有存在有效内容且结构能改善理解或操作时，栏目才可出现。	14-自然语言呈现与信息密度规范
TXT-STRUCT-002	禁止生成空的“背景”“风险”“下一步”“相关资料”等标题。	14-自然语言呈现与信息密度规范
TXT-STRUCT-003	简单 Task 可以只有一段文字和隐藏属性；不得为了符合 Project 模板而膨胀。	14-自然语言呈现与信息密度规范
TXT-STRUCT-004	结构深度默认不超过三级。更深细节应通过折叠、关联页或文件工作区承载。	14-自然语言呈现与信息密度规范
TXT-STYLE-001	主体内容 SHOULD 使用连贯自然语言解释因果、进展和下一步；只有当项目适合比较、枚举或执行时才使用列表。	14-自然语言呈现与信息密度规范
TXT-STYLE-002	不得把完整信息机械拆成“时间、地点、人物、目的、过程、结果”属性清单，除非用户正在查看属性面板或该格式具有明确操作价值。	14-自然语言呈现与信息密度规范
TXT-STYLE-003	语言应具体、克制、可验证。避免“全面赋能”“进一步提升”等没有事实含量的表述。	14-自然语言呈现与信息密度规范
TXT-STYLE-004	正文应区分：已确认事实、当前判断、推测、建议和待确认问题。	14-自然语言呈现与信息密度规范
VIEW-BASE-001	每个视图必须声明用户目标、数据范围、排序、显示字段、隐藏字段和高频操作。	13-注意力场景与视图投影规范
VIEW-BASE-002	同一对象在多个视图中必须引用同一 object_id，所有修改回到同一权威状态。	13-注意力场景与视图投影规范
VIEW-CUS-001	自定义视图只保存查询和投影，不得改变对象归属。	13-注意力场景与视图投影规范
VIEW-CUS-002	Logseq Query 可以作为一种适配实现，但领域视图定义不得绑定某一种查询语法。	13-注意力场景与视图投影规范
VIEW-RE-001	重入包必须以恢复行动为目标，不得只是把所有事件压缩成一篇长摘要。	13-注意力场景与视图投影规范
VIEW-WAIT-001	Waiting 和 Blocked 必须分开呈现，因为前者主要需要复查，后者主要需要解除阻碍或重新决策。	13-注意力场景与视图投影规范